

Molarium: Agent-Generated Adaptive Software for Molecular Design

A perspective on user-directed adaptive software, persistent molecular state, and scientific reproducibility in drug discovery

Rafal P. Wiewiora PsiThera rafal.wiewiora@psithera.com	W. Patrick Walters OpenADMET pat.walters@omsf.io	Woody Sherman PsiThera woody.sherman@psithera.com
---	---	--

September 2, 2026

Abstract

Commercial drug discovery software has traditionally bundled three sources of value: the interface, the scientific models, and the specialized expertise required to run the models through the interface, thereby translating outputs into design decisions. Large language models, coding agents, and tool-using agents are beginning to unbundle this paradigm. Interfaces, analyses, workflow logic, and substantial portions of scientific software can increasingly be generated to meet the needs of a particular scientist, project, and question, while agents can automate sophisticated workflows that once required specialist users.

This shift repositions where competitive advantage is created in computational drug discovery. More intuitive interfaces and faster execution alone do not fix model inaccuracies or broaden the applicability domain. As the underlying tooling becomes increasingly accessible and fluid, strategic differentiation will shift away from familiarity with a particular software package and toward superior predictive algorithms, more generalizable models, proprietary experimental data, accumulated discovery intelligence, and the human discernment needed to frame the right questions and evaluate the answers.

Molarium, available at <https://molarium.org>, provides a case study of this transition in the context of molecular design software. Created during an intensive weekend effort of a tireless drug-discovery scientist and an AI coding agent, this open-source molecular workbench operates exclusively in a standard web browser. It integrates local molecular graphics, editing, conformer generation, energy minimization, molecular dynamics, machine-learned potentials, and protein-structure prediction workflows. Beneath its rapidly adaptable interface, Molarium maintains a versioned molecular state together with user actions, method specifications, provenance, and validation evidence. Decoupling an adaptable interface from an enduring scientific core points toward a broader architecture for generated adaptive software in drug discovery.

Keywords: agentic software development; computational chemistry; molecular design; WebGPU; scientific validation; provenance; discovery intelligence

1 From fixed software products to generated adaptive software

In computational drug discovery, success depends on deploying the right methodology at the right time, supported by appropriate parameters, explicit assumptions, and sound scientific context. Historically, this work has been concentrated among computational specialists and has required substantial operational overhead. Docking depends on protein preparation, protocol selection, constraints, and pose evaluation. Molecular dynamics requires complex system setup, solvation, ion placement, force-field assignment,

compute infrastructure, equilibration, production simulation, potential for enhanced sampling, trajectory analysis, visualization, and interpretation in the context of project-specific decisions. Free-energy calculations add alchemical perturbations, sampling windows, convergence assessment, error analysis, and alignment with experimental conditions. Comparable layers of infrastructure surround cheminformatics, quantum chemistry, ADME property prediction, generative molecular design, and protein structure prediction.

Historically, commercial scientific software has created considerable value by simplifying and integrating many of these activities. Graphical interfaces reduced technical barriers, workflow engines connected fragmented programs, and software packages encoded operational details needed to apply methods consistently. A single static interface (perhaps updated quarterly) coupled scientific questions to a fixed software environments. User-specific requests competed with broader product roadmaps that attempted to balance the needs of a diverse users base. Implementing new methods from the literature or integrating open-source software often required software engineering support, if even possible. Scientific users frequently adapted their questions to the capabilities and conventions of the available tools within their preferred/available graphical interface.

Large language models and coding agents are beginning to change this relationship. A scientist with deep drug-discovery expertise but limited software programming experience can increasingly describe an application, analysis, visualization, or workflow in natural language and have software constructed around the scientific need. Tool-using agents can operate established methods, build complex workflows, implement methods from the literature, handle failures, assemble results into forms that support decisions, and even optimize processes based on iterative feedback. Earlier systems such as ChemCrow and ChemGraph established proof-of-concept for language models coordinating chemistry tools and computational workflows [1–4]. While their practical scope was limited compared with current coding agents, they helped establish the feasibility of coupling language models to executable scientific tools.

The consequence is a shift in how drug hunters can interact with software. Scientists can increasingly customize software around project-specific questions, available data, and their preferred way of reasoning about a problem. A medicinal chemist optimizing a macrocycle may need a different interface and set of calculations than a structural biologist testing a mechanistic hypothesis or a program team deciding which compounds to advance from a virtual screening campaign. In this model, the interface becomes an adaptive and potentially transient view over a more durable scientific substrate. Changes to the interface do not imply changes to the underlying record of the work: molecular state, computational methods, parameters, intermediate results, user actions, and software versions are all persisted and linked through an auditable provenance history. This separation between a mutable presentation layer and a persistent scientific state is important for reproducibility, inspection, and reconstruction of prior decisions. Although Molarium presents as a relatively simple browser-based application, it is built on a substantially richer underlying architecture for managing versioned molecular state, tracking computational provenance, and executing adapted implementations of established molecular modeling algorithms efficiently within the constraints of a local web browser.

Molarium’s development directly reflects this paradigm. It began as a need for a better environment to view and modify molecular structures, then evolved through active scientific use. Scientific problems drove software changes: unrealistic geometries led to refined editing semantics; questionable energies prompted explicit numerical reference checks; ambiguity about execution backends led to clearer identification of the active engine; and unsupported methods became explicit boundaries rather than implicit assumptions. The software evolved with the scientific questions it was built to address. Appendix ?? describes the execution architecture, numerical validation, and an executable example of this design history. Appendix ?? provides a module-by-module reference distinguishing current, experimental, repository-only, proposed, and retired functionality.

2 Molarium and the value of a persistent molecular state

Molarium is an MIT-licensed, local-first molecular viewer, builder, and simulation workbench. Most supported calculations execute on the user’s device using WebAssembly or WebGPU, allowing molecular graphics, editing, cheminformatics, conformer exploration, molecular mechanics, molecular dynamics, machine-learned potentials, and selected protein-structure prediction workflows to coexist in a standard browser environment. The browser reduces installation and orchestration overhead and, for supported calculations, can avoid remote job submission, cloud-compute charges, and application licensing while keeping computation inside the interactive design loop. WebAssembly provides a route for mature compiled scientific libraries to run locally, while WebGPU exposes modern accelerators through a portable browser API [5, 6].

The architectural choice that matters most is the preservation of a common scientific state. Molarium maintains explicit topology, coordinates, protonation state, chemical edits, preparation history, trajectories, model identity, and provenance as properties of the molecular session rather than allowing each view or calculation to construct its own interpretation of the molecule. The interface can therefore change rapidly without redefining the scientific object beneath it.

Three task-specific views illustrate this separation. View emphasizes the molecular system, including protein, ligand, waters, ions, contacts, and selected residues. Design exposes chemical and geometrical interventions. Simulate emphasizes preparation, trajectories, and analysis. The controls differ because the scientific questions differ; topology, coordinates, method identity, and history remain attached to the same evolving molecular state. Detailed implementation and replay examples are provided in Appendix A, while the state mutations, exports, and failure behaviour of the corresponding user-facing modules are specified in Appendix B.

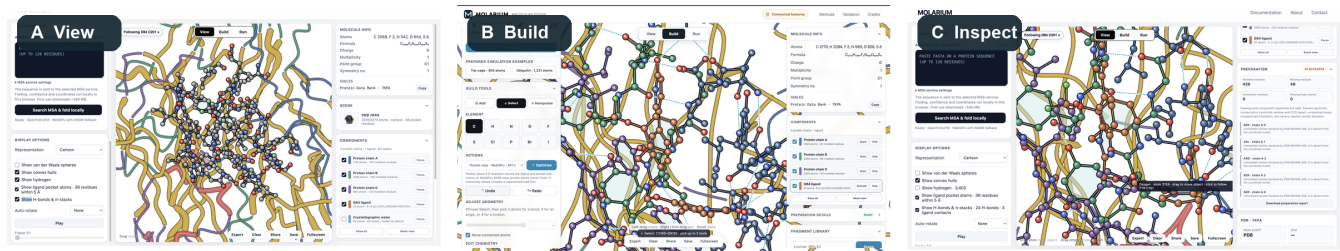


Figure 1: Molarium as an adaptive view over a persistent scientific state. Different views emphasize different design questions while the underlying molecular state, topology, coordinates, history, provenance, and computational context remain connected.

This separation becomes important when interfaces are inexpensive to modify. If every generated view can redefine a molecule, select a different charge model, change protonation, alter preparation, or reinterpret a trajectory, flexibility becomes a source of scientific ambiguity. Generated adaptive software therefore requires a stronger boundary between presentation and scientific state than a conventional fixed interface typically provides.

Molarium extends this principle by preserving the sequence of molecular states and user actions that produced the current session. Edits, selections, manipulations, calculations, and other interactions form an event history rather than leaving only the final coordinates. The resulting architecture is closer to version control than to a traditional molecular project file. Each molecular state can be associated with the actions that produced it, the calculation applied to it, and the evidence available at that point in the design process.

The “Git for molecules” analogy is useful, although a drug-design history is richer than source code. A design trajectory includes structural changes together with evolving hypotheses, model outputs, uncertainty, and experimental evidence. In a discovery setting, such a record could capture which analogs were

considered, which structures or properties influenced a design, which alternatives were rejected, and what was learned when a compound was synthesized and tested. This creates a foundation for recording medicinal-chemistry design trajectories rather than storing only the compounds that survived them. Much of the information that explains how a program learned lies in alternatives that were considered and abandoned, yet conventional compound databases usually preserve endpoints.

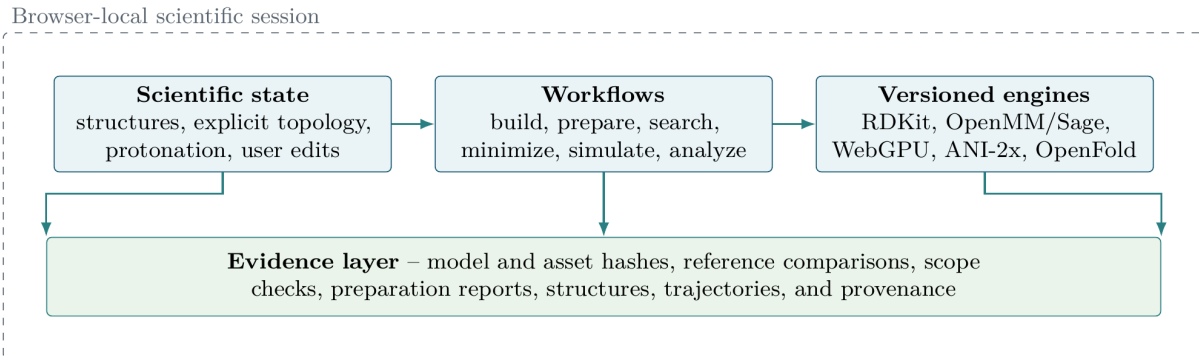


Figure 2: Molarium’s local-first architecture. Named computational engines act on an explicit molecular state while provenance and validation evidence remain attached. The same principle provides a foundation for versioned molecular histories in which scientific actions can be replayed and compared without allowing a new interface to redefine the underlying state.

The browser also provides a useful trust boundary for proprietary work. Molarium’s Local Lab mode can restrict network access and verify reviewed application assets while keeping supported calculations on the local device. Because privacy controls depend on deployment configuration, this architecture makes data movement explicit and auditable. Organizations can evaluate and enforce how molecular data are handled rather than treating transfer to remote infrastructure as an invisible property of the software stack. Appendix B specifies the connected and Local Lab network policies and makes clear that local calculation and fully offline operation are distinct claims.

3 Scientific contracts: provenance, validation, and falsifiability

The emergence of cheap, adaptive software introduces a scientific vulnerability. Agentially generated workflows become especially risky when seamless execution obscures the boundaries of the underlying scientific contract. As the time required to build a custom analysis view drops to minutes, that view still needs to expose the molecule, model, parameters, assumptions, and evidence that produced the result. Reducing the friction of software development without those safeguards increases the rate at which scientific ambiguity can be created.

We use the term scientific contract for the collection of invariants and evidence that gives a computational output its meaning. A molecular-mechanics result cannot be defined by the generic label “molecular mechanics.” Its identity depends on the molecular state, charge assignment, parameterization, energy function, treatment of nonbonded interactions, boundary conditions, constraints, integrator, numerical implementation, and precision. A learned potential depends on its architecture, weights, preprocessing, supported chemistry, and training domain. A protein-structure prediction depends on checkpoint selection, sequence and template information, inference protocol, and other model-specific choices. The surrounding interface may adapt dynamically, but these scientific identities must remain explicit and protected from silent drift.

Molarium preserves distinctions among computational pathways even when they operate in the same workspace. RDKit supports chemical perception and conformer generation; OpenFF Sage defines a molecular-mechanics model; OpenMM Reference provides an independent numerical route for selected

validation tests; STORMM-inspired WebGPU code advances homogeneous replicas; ANI-2x provides a machine-learned potential with a defined chemical domain; and OpenFold supports a separate protein-structure prediction pathway [7–12]. These methods can be compared or chained without implying that their energies, assumptions, or domains are interchangeable. Appendix A specifies the implemented numerical pathways and their validation boundaries; Appendix B records how those pathways are exposed, invoked, and constrained in the current workbench.

The same principle applies to workflow provenance. Reproducible computational science requires retention of the data, computational process, execution environment, and provenance that produced a result [15, 17]. More recent work formalizes workflow invariants and testable reproducibility criteria rather than treating provenance as passive metadata [16]. Generated adaptive software increases the importance of this requirement because the workflow itself may be created or modified during the scientific interaction rather than existing as a stable artifact defined in advance.

This leads to a stronger requirement: generation of a scientific workflow should be accompanied by generation of the conditions under which its claims could be falsified. If an agent implements a conformer-search strategy, the output should include the benchmark that tests whether conformational coverage improves, the independent reference used for comparison, the quantitative tolerance that defines agreement, the applicability limits of the test, and explicit failure conditions. If an agent adds a numerical kernel, it should generate the corresponding independent comparison and acceptance criteria. These are better understood as scientific contract tests than as ordinary software unit tests because they define what scientific claim the implementation can support.

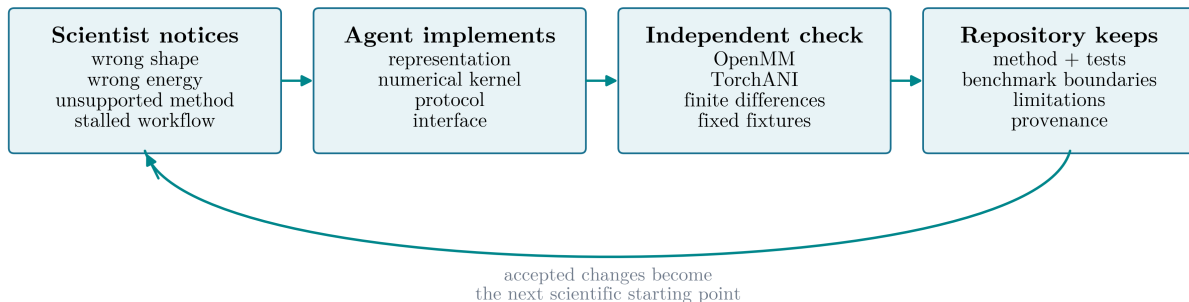


Figure 3: The scientist-agent build loop. Scientific needs and objections are translated into software changes, while each new capability is paired with claim-defining tests, reference evidence, applicability limits, and provenance. The ability to generate a workflow and the ability to specify how its claim could fail are treated as parts of the same scientific result.

Molarium’s development repeatedly followed this pattern. A chemically implausible geometry exposed a problem in representation and editing semantics. A questionable energy prompted comparison with an independent implementation. A discrepancy between CPU and GPU execution revealed both a unit error and an incorrect assumption about which backend was running. Requesting constrained dynamics led to implementation of SHAKE/RATTLE and comparison against a reference pathway [13, 14]. In each case, the useful scientific result consisted of the feature together with a test that established the boundary of the claim.

Fluent agentic software makes this distinction more consequential. A unit error can produce a smooth trajectory. Separate implementations can agree if they inherit the same flawed inputs. A numerically correct implementation can apply an inappropriate physical model. Successful execution and an attractive visualization therefore occupy the bottom of an evidence hierarchy rather than establishing that a molecular prediction is correct.

When software construction is difficult, the labor of building a computational method can obscure the

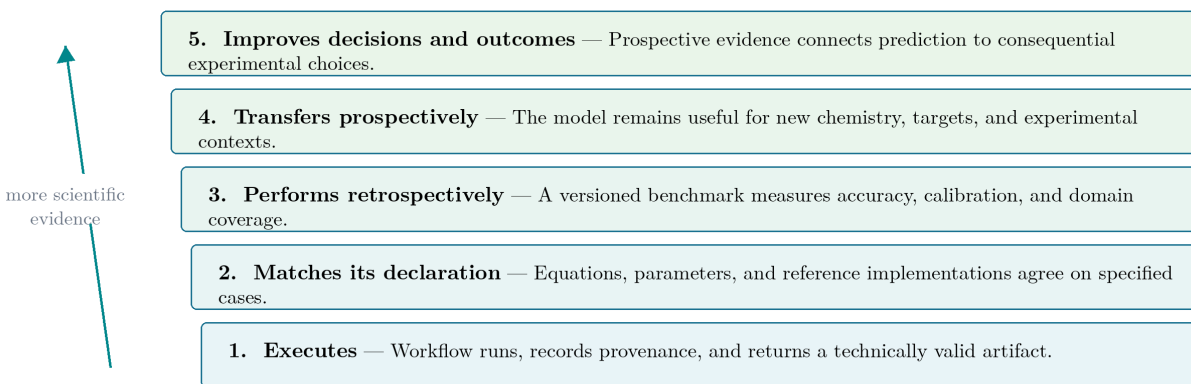


Figure 4: An evidence ladder for generated scientific software. Successful execution establishes less than numerical agreement; numerical agreement establishes less than retrospective predictive performance; retrospective performance establishes less than prospective transfer; and prospective accuracy is distinct from measurable improvement in drug-discovery decisions.

separate obligation to validate it. As implementation becomes easier, the scientific responsibility becomes harder to ignore. The workflow, its provenance, and the conditions that could falsify its claims should therefore be treated as a single scientific object.

4 Local computation changes the interaction model

A web browser does not replace production molecular simulation software or high-performance computing. Large periodic systems, long trajectories, advanced sampling, specialized force fields, and large-scale parallel studies can require substantially more compute than a personal machine can provide. Molarium addresses a different part of the workflow: calculations that are sufficiently bounded to remain inside the scientist’s interactive design session.

The practical benefit is reduced decision latency. Consider a medicinal chemist examining a bound ligand and asking whether a proposed three-dimensional edit creates an avoidable steric problem or opens a plausible alternative conformation. In a fragmented workflow, the scientist may export coordinates, request or run a separate calculation, move results back into a viewer, and reconstruct the context in which the question arose. In Molarium, supported editing, local relaxation, conformational exploration, and trajectory inspection can occur in the same session. The value comes from keeping the molecular state, the calculation, and the design question connected while the decision is still active.

Local execution also changes economics for these interactive calculations. The marginal cost of many small calculations approaches the cost of hardware already on the user’s desk, without requiring a remote orchestration cycle or a separate software license for each supported operation. This advantage is most relevant in the rapid design loop, where a calculation may be useful because it answers a focused question in minutes rather than because it achieves the maximum possible throughput on a benchmark.

The lower operational barrier increases the importance of choosing calculations deliberately. A medicinal chemist deciding between two substituent orientations may gain value from a constrained local refinement that changes which analog is synthesized next; running hundreds of additional simulations after the decision is already insensitive to the result adds computational activity without increasing scientific leverage. The operational gain matters when it helps the scientist focus on the right question, apply the appropriate method, and prioritize molecules that have a realistic chance to advance the program.

Local execution also redistributes control. The scientist can determine which analysis should exist, how data should be juxtaposed, and which calculation should be available at the point of decision. Molarium

illustrates this shift: a scientist did not need to wait for every useful workflow to appear on a conventional product roadmap. Missing interfaces and bounded implementations could be added while the underlying question was still active.

5 What becomes scarce when the tool layer becomes cheap

A concrete medicinal-chemistry example helps separate the layers of value. Suppose a project team is considering a substitution intended to improve a ligand’s interaction with a newly identified pocket while preserving permeability. The tool layer provides the editor, visualization, coordinate handling, workflow logic, and connection to the relevant calculations. The model layer determines whether the predicted pose, conformational preference, interaction energy, or permeability estimate is credible. The discovery-intelligence layer supplies the project-specific context: prior SAR, assay conditions, structural interpretation, known liabilities, synthetic constraints, and the reason the team is considering that particular design at that point in the program. Coding agents can make the first layer easier to construct. The value of the second and third layers depends on scientific accuracy and accumulated knowledge rather than interface complexity.

The tool layer includes graphical interfaces, visualization, format conversion, wrappers, workflow logic, reports, job control, data plumbing, and bounded numerical implementation. These capabilities remain important and require maintenance, but their marginal reproduction cost is falling quickly as coding agents improve. A scientist can increasingly generate a project-specific view or connect an existing computational method without waiting for the corresponding feature to appear in a commercial release.

The model layer includes force fields, quantum-mechanical methods, docking and scoring functions, free-energy models, machine-learned potentials, protein-structure predictors, molecular representations, property models, parameters, weights, uncertainty calibration, and the evidence that establishes their useful domain. Easier invocation does not improve predictive quality. A docking model with an inadequate scoring function remains inadequate when an agent invokes it. A systematic force-field error can become more damaging when automated workflows propagate it across thousands of calculations. A machine-learned property model does not gain extrapolative validity because a language model describes its output fluently, and a protein-structure predictor does not become reliable for a particular conformational state because it is available through a better interface.

Differences among models therefore become more consequential as the mechanics of invoking them become less scarce. Drug discovery frequently requires extrapolation into new chemistry, structural states, and biological contexts rather than interpolation within a well-sampled training distribution [18–20]. Faster access can accelerate good models and weak models alike.

The discovery-intelligence layer includes proprietary experimental data, negative results, project history, assay context, structural interpretation, design rationale, internal protocols, expert judgment, and program objectives. Molarium intentionally contains little of this layer because it is a public workbench built primarily from public scientific components. The same architecture could be connected to internal structures, assay data, pharmacokinetics, target biology, medicinal-chemistry reasoning, and program objectives. That project-specific context can determine which computation is relevant and how much weight its output should receive.

Versioned molecular history provides a bridge into discovery intelligence. Traditional compound databases record what was made and measured well, but they rarely preserve what was considered, why one design was preferred, which model or structure influenced the decision, or what evidence was visible at the time. A persistent action history can capture that decision trajectory.

For example, a chemist might propose adding a substituent after inspecting a pocket in a protein–ligand structure. The project record could retain the parent structure, the molecular edit, the structural hypothesis, the conformational or energetic calculations viewed before synthesis, competing designs that were rejected,

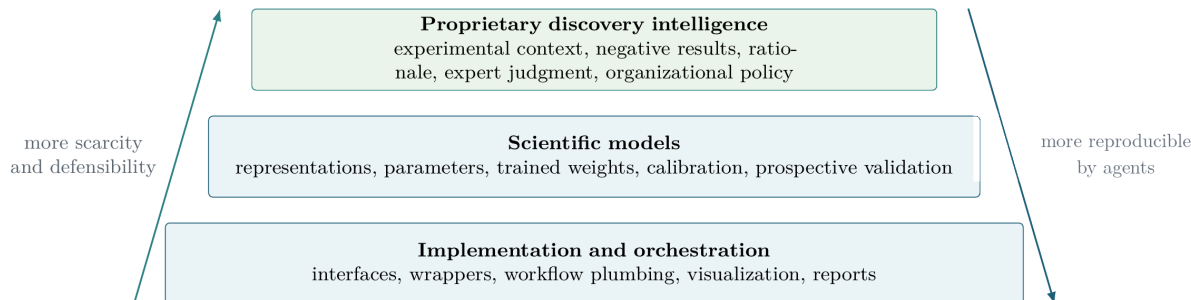


Figure 5: Three interdependent layers of value in generated adaptive molecular design. Implementation and orchestration are becoming increasingly reproducible by coding agents. Scientific models remain differentiated by accuracy, representation, domain coverage, calibration, and validation. Proprietary discovery intelligence combines experimental data with the context and decision history that make those data useful.

and the subsequent experimental measurements. If the design later fails because an assumed interaction was absent or a liability dominated the profile, that information remains associated with the decision that created the molecule. Future scientists and agents can then ask which assumptions repeatedly led to productive designs and which repeatedly failed. This is decision provenance: preserving the relationship among hypotheses, interventions, predictions, decisions, and outcomes rather than storing only final compounds and assay values.

Agentic systems provide a mechanism to operationalize some of this accumulated knowledge through scientific skills. Here, a skill means a version-controlled, machine-executable procedure that combines a defined tool or model with an organization’s accepted inputs, assumptions, validation criteria, applicability limits, failure handling, and escalation rules. A protein-preparation skill might encode how crystallographic waters, protonation states, alternate conformations, unusual residues, and ambiguous cases are handled. A free-energy skill might define accepted perturbations, convergence criteria, diagnostics, and failure modes. The durable asset is the scientific procedure and its validation, rather than the natural-language prompt used to invoke it.

6 The changing role of scientific expertise

As software becomes easier to construct and operate, scientific judgment becomes more valuable. Traditional computational expertise includes substantial operational knowledge: preparing inputs, locating settings, translating files between programs, recovering failed jobs, and reproducing standard analyses. An increasing fraction of this work can be encoded in software and agents. The broader automation literature has long recognized that reducing routine operation can increase the importance of human expertise when systems encounter ambiguous or abnormal conditions [21].

The highest-value expertise shifts toward selecting the right question, choosing an appropriate level of model fidelity, defining acceptable tradeoffs between cost and accuracy, recognizing implausible results, selecting meaningful controls, and determining which evidence would change a project decision. Scientists closest to the underlying discovery problem—whether medicinal chemists, structural biologists, mechanistic biologists, pharmacologists, or computational scientists—can increasingly interact directly with relevant data and calculations rather than routing every question through a software intermediary.

This shift changes the leverage of computational specialists. Less specialist time needs to be spent constructing repetitive interfaces, transferring files, or manually operating routine workflows. More can be spent on model development, molecular physics, parameterization, sampling, uncertainty, validation, and difficult edge cases where established workflows fail. Once that expertise is made explicit, part of it can be

encoded into validated scientific skills that broaden access without implying that the underlying science has become simple.

Molarium’s development illustrates this division of labor. The coding agent could inspect a repository, refactor software, implement numerical kernels, add tests, and repeat the loop rapidly. The scientist had to recognize chemically implausible results, decide whether a reference was genuinely independent, determine whether a discrepancy mattered, choose acceptable approximations, and identify what evidence would change a decision. The agent compressed the distance between recognizing a scientific need and changing the software; scientific judgment determined which changes mattered.

The bottleneck therefore moves toward epistemology: deciding which approximation is acceptable, whether a faster method sacrifices information that matters to the decision, when an independent control is required, whether a reference is truly independent, whether a model is operating outside its validated domain, what uncertainty must remain visible, and which experiment would discriminate between competing hypotheses. Faster implementation increases the value of asking the right question.

7 Limitations and outlook

Molarium is a case study rather than a controlled experiment in software productivity or drug-discovery performance. Its development involved a small number of scientists, one evolving codebase, and a particular coding-agent workflow. The project did not include a randomized comparison with a conventional software-development team, and it cannot establish a universal productivity multiplier or determine what fraction of commercial scientific software can be regenerated reliably.

The open-source repository can nevertheless support a useful community experiment. Contributors can identify computational capabilities described in papers or public specifications, attempt independent agent-assisted reconstructions, test them against appropriate references, and record which parts can and cannot be reproduced reliably. Over time, this could become a living benchmark for the regeneration of molecular-modeling software, with success defined by scientific contract tests rather than by code generation alone.

Molarium’s scientific scope is deliberately bounded. Current browser-native molecular mechanics does not reproduce the full capability of mature native simulation environments. General periodic simulation, particle-mesh Ewald electrostatics, barostats, membranes, unrestricted protein parameterization, arbitrary plugins, and other advanced capabilities remain outside the current browser-native scope. ANI-2x remains valid only within its declared chemical domain, and browser-based OpenFold support is narrower than general production protein-structure prediction workflows. Generated wrappers and easier access must preserve, expose, and enforce these applicability limits rather than implying broader scientific validity. Appendix A catalogues the demonstrated implementation and numerical boundaries, and Appendix B provides the corresponding operational status, failure modes, and end-user limits.

A second limitation concerns the distinction between implementation fidelity and predictive validity. If a browser implementation reproduces an OpenMM reference energy and force to a specified tolerance, that result establishes that the implementation is executing the declared calculation correctly for the tested case. It does not establish that the shared force field predicts the experimental behavior of a new molecule. Likewise, a conformer workflow can reproduce its intended algorithm and explore more structures without improving recovery of biologically relevant conformations. Implementation validation and model validation answer different questions and require different evidence.

The future suggested by Molarium is one in which the interface between scientist and computation becomes more fluid while the scientific state, models, assumptions, evidence, and decision history become more explicit. Project teams can construct temporary views around specific questions; internal models can coexist with open-source methods and commercial services; and the user experience no longer needs to be owned by the provider of each underlying calculation.

The durable infrastructure in such an environment is concrete. A user should be able to inspect the active molecular state and its history, the exact model and parameter versions used for each calculation, declared applicability limits, validation status, assumptions, provenance, and links between predictions and subsequent decisions. These quantities should remain available regardless of which generated view is open. An adaptive interface can change rapidly because the scientific contract underneath it remains stable and inspectable.

8 Conclusion

Molarium began as a request for a better environment in which to inspect and manipulate molecules. It developed rapidly into a local-first molecular-design workbench incorporating topology-aware editing, protein preparation, conformer exploration, molecular mechanics, molecular dynamics, learned potentials, protein-structure prediction, browser-native GPU computation, provenance, and reference validation. The speed of that development is technically interesting, but the broader lesson concerns the architecture of scientific software and the location of durable value.

Software can increasingly be generated around the scientific question and the user’s specific preferences. The quality of the underlying models, validation, and decision history becomes more consequential as the interface becomes cheaper and more fluid. The scarce assets shift toward trustworthy models, proprietary discovery intelligence, and the human judgment required to know what to ask and what to believe.

Molarium also suggests that molecular-design environments should preserve more than the latest structure. A versioned record of molecular states, user actions, calculations, assumptions, and results can capture the trajectory by which scientific decisions were made. Coupled to explicit scientific contracts and falsification tests, that history can make generated adaptive software more reproducible, extensible, and scientifically accountable.

When software is difficult to construct, implementation effort can hide the distinction between building a method and validating it. When implementation becomes easy, the remaining scientific obligations become harder to ignore. The opportunity is to place adaptable computational capability directly in the hands of scientists who understand the problem while preserving the models, evidence, and decision history that determine whether the answer should be trusted.

The tool layer is becoming cheap and adaptive. Scientific advantage moves toward better models, better discovery intelligence, and better decisions.

AI assistance and availability

GPT-5.6 Sol was used extensively as a coding agent during the development of Molarium under human scientific direction, inspection, testing, and revision. Language-model assistance was also used in restructuring and drafting this manuscript. The authors remain responsible for the scientific claims, validation, source attribution, and final text.

Molarium is available at <https://molarium.org>. Its source code and technical documentation are publicly available at <https://github.com/rafwiewiora/molarium>. Quantitative benchmark results are scoped by the relevant implementation state, fixtures, model and asset hashes, browser and hardware environment, numerical precision, and timing protocol. Appendix A provides the architecture, numerical validation, and executable design-history evidence underlying these results; Appendix B records the inspected operating surface, module boundaries, failure behaviour, and export/persistence contracts.

APPENDIX A

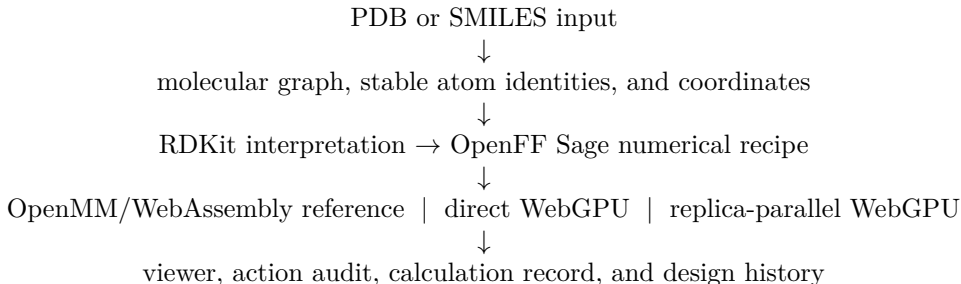
Molarium architecture, numerical validation, and executable design history

This appendix specifies the implementation and validation substrate underlying the architectural claims in the main text. It defines the molecular-state contract shared across chemistry perception, parameterization, numerical execution, visualization, and provenance; describes the OpenMM/WebAssembly and WebGPU execution paths; reports fixed-input numerical validation and workload-dependent performance measurements; and documents the content-addressed replay and decision-history mechanisms used by Molarium. The final section provides an executable SOS1 design-history example. Numerical agreement is treated as implementation validation of a declared calculation and is kept distinct from validation of the underlying physical or statistical model.

A.1 Execution architecture

Molarium maintains a single molecular state keyed by stable atom identifiers across ingestion, chemical perception, parameterization, calculation, visualization, and replay. RDKit provides graph interpretation, conformer generation, SMARTS matching, and ligand-level preparation. A JavaScript SMIRNOFF parameterizer applies OpenFF Sage 2.1.0 to generate an explicit numerical recipe containing particle properties, bonded terms, nonbonded terms, exceptions, and constraints. Preparation fails on missing required parameters rather than substituting inferred terms. The accepted recipe is consumed by the OpenMM/WebAssembly reference path and the direct or replica-parallel WebGPU paths, allowing backend comparisons on identical topology, coordinates, parameters, and atom ordering.

The primary data path is:



The same molecular object remains authoritative as calculations return coordinates, forces, energies, or trajectories. Table 1 summarizes the current implementation.

Internal units are nm, ps, daltons, kJ/mol, and kJ/mol/nm; the interface presents angstroms and kcal/mol where appropriate. Numerical engines execute in reusable Web Workers so rendering and user interaction remain responsive. Requests carry job identifiers, and failed workers are discarded and recreated. Results return as compact coordinate, force, energy, or trajectory arrays mapped to the originating atom identities.

For ordinary small molecules, the browser parameterizer applies Sage 2.1.0 patterns and labelled Gasteiger charges [8, 26]. Prepared Rosemary validation fixtures retain their exported numerical recipes and NAGL charges. CPU/GPU comparisons therefore operate on the same prepared graph and numerical inputs rather than independently parameterized systems.

A.1.1 OpenMM WebAssembly reference

OpenMM 8.2.0 is cross-compiled with Emscripten 6.0.6-git and CMake 3.31.0 [9, 27]. A thin C interface constructs an OpenMM `System` from the serialized numerical recipe and returns energies, forces, minimized coordinates, or short trajectories. Source, patches, interface code, and built assets carry recorded fingerprints so the deployed reference path can be associated with a specific implementation state.

Table 1: Current browser implementation.

Task	Software used			Implementation role
Molecular interpretation	RDKit	2025.03.4,	WebAssembly	SMILES/graph handling, ETKDGV3, SMARTS matching, MMFF94, explicit UFF fallback, and Gasteiger charges
Force-field assignment	OpenFF	Sage 2.1.0		Generates particles, bonds, angles, torsions, nonbonded terms, exceptions, and constraints
Numerical reference	OpenMM	8.2.0	Reference, WebAssembly	Double-precision energies and forces, L-BFGS minimization, and short dynamics used for fixed-input reference comparisons
Direct local mechanics	WebGPU/WGSL,	32-bit floating point		Single-system force/energy evaluation, OBC2/ACE, minimization, and short dynamics
Replica-parallel mechanics	STORMM-inspired	WebGPU engine		Advances independent replicas sharing one topology and parameter layout; paired-word fixed-point accumulation; current limit 512 atoms per replica
State and provenance	JavaScript modules, background workers,	Canvas, Web Crypto		Preserves atom identity and records action, calculation, replay, and SHA-256-linked provenance

The reference configuration is nonperiodic and uses the double-precision OpenMM Reference platform. Supported operations include energy and force evaluation, L-BFGS minimization, short dynamics, optional X–H constraints, 2 fs Langevin-middle integration, a cutoff configuration used for validation, and OBC2/ACE with mbondi2 radii [28].

The WebAssembly build exposed a threading-specific setup failure for constrained dynamics: native OpenMM creates helper CPU threads during constraint initialization, whereas the browser build is single-threaded. A serial patch removes the wait path for this build without modifying native OpenMM.

Fixed-input parity was evaluated in two stages. The same C interface compiled natively and for WebAssembly agreed to a maximum energy difference of 2.84×10^{-14} kcal/mol and relative force RMS of 3.76×10^{-15} . Five hash-selected protein–ligand poses were then prepared through browser Sage and evaluated with matched numerical inputs. Maximum energy differences were 1.24×10^{-4} kcal/mol in vacuum and 2.76×10^{-4} kcal/mol with OBC2; maximum relative force RMS error was 1.16×10^{-5} in both modes. These tests establish implementation parity for the specified fixtures and protocols.

A.1.2 Direct WebGPU mechanics

The direct WebGPU path evaluates the prepared force-field equations in WGSL using 32-bit floating-point arithmetic. Force ownership is atomwise: one invocation owns the output force for one atom, and compact incidence lists identify the bonded terms that contribute to that atom. Pair and valence terms are recomputed where required so each invocation writes its force once, avoiding floating-point atomic updates, which WGSL does not provide [22].

The deployed nonbonded representation stores excluded and specially scaled pairs explicitly and mixes ordinary Lennard–Jones parameters on demand. An earlier all-pairs metadata representation required $32N^2$ bytes and imposed a practical allocation ceiling; the sparse exception representation removes that quadratic metadata cost. OBC2/ACE is evaluated in three kernel passes for Born radii, coordinate derivatives, and solvent/nonpolar energy and force accumulation. Optional X–H constraints are graph-coloured so non-overlapping SHAKE/RATTLE corrections can execute concurrently [13, 14]. Seeded Langevin dynamics supports 1 fs flexible and 2 fs constrained integration.

Pocket relaxation uses a bounded steepest-descent procedure with zero inverse mass on fixed receptor atoms; only the ligand and explicitly selected pocket atoms move. Current operating limits are listed in Table 2.

At the interactive system sizes tested here, a conventional Verlet neighbour list did not improve performance. With a 0.2 nm skin rebuilt every 20 steps, 5,000 constrained OBC2 Trp-cage steps were 46% slower and 1,000 ubiquitin steps were 17% slower than full-range nonbonded evaluation on an Apple M1 Pro. The cutoff route is therefore retained for validation but is not exposed in the product controls.

GPU benefit depends on amortizing dispatch and data movement. A single Trp-cage energy/force evaluation required 25.5 ms in WebGPU and 5.7 ms in OpenMM Reference. Across 1,000 dynamics steps, WebGPU reached 214.9 versus

Table 2: Operating limits in the current direct WebGPU worker.

Setting	Current value
Principal force workgroup	64 atomwise calculations
Time step	1 fs flexible; 2 fs with X–H constraints
Constraint solve	Four graph-coloured sweeps by default; at most 32
Minimization	750 iterations by default; at most 2,000; each displacement capped at 0.001 nm
Direct dynamics request	At most 5,000 steps; longer requests use the ensemble route
Validation cutoff	1.0 nm cutoff; 0.2 nm skin; rebuild every 20 steps

102.8 ns/day for 304-atom Trp-cage and 65.18 versus 7.11 ns/day for 1,231-atom ubiquitin. These measurements use warmed, nonperiodic, all-pairs workloads. The direct path does not currently implement periodic boundaries, particle-mesh Ewald electrostatics, explicit solvent, barostats, virtual sites, or arbitrary OpenMM custom forces.

A.1.3 Replica-parallel WebGPU

The replica engine targets throughput across independent copies of a shared topology. One replica maps to one GPU workgroup, in contrast to the atomwise decomposition used by the direct engine. The memory layout and fixed-point accumulation strategy draw on STORMM’s stacked-system design, while the browser implementation is independent WGS� code [10]. Replicas share topology and parameters but do not interact.

Concurrent accumulation uses paired 32-bit words to represent signed 64-bit fixed-point values because WGS� exposes 32-bit integer atomics. The implementation uses 2^{22} counts per kcal/mol for energies, 2^{18} counts per kcal/mol/Å for forces, and 2^{32} counts per Å for coordinates. This replaced 32-bit accumulators that overflowed silently in highly clashed configurations and produced bit-for-bit replay for identical seeds on the tested adapter.

Integration uses velocity Verlet, seeded Langevin thermalization, and SHAKE/RATTLE. Constraint projection is serial within a replica while replicas execute in parallel; the default relative tolerance is 10^{-5} with at most 32 iterations. Commands contain at most 50 steps. Persistent status words convert non-finite values, accumulator overflow, coordinate clamps, and failed constraints into terminal errors. Current limits are 512 atoms per replica, all-pairs nonbonded evaluation, and identical atom count, topology, and parameters across replicas.

Force evaluation uses a separate 32-bit floating-point coordinate working copy. Translating a test system 500 Å from the origin changed energies by approximately 3×10^{-4} relative, identifying the working coordinate representation rather than the fixed-point store as the remaining translation-sensitivity boundary. Eliminating this source of error requires recentering before force evaluation or a wider working representation.

Replica throughput also crosses over with workload size. In warm tests, OpenMM Reference was 13.3-fold faster for one C16 molecule and 3.5-fold faster for one 27-water system. WebGPU was 10.3-fold faster for 1,024 C16 replicas and 24.1-fold faster for 256 water replicas. These measurements compare aggregate replica throughput; the two engines do not use identical integrators.

A.2 Reference-driven implementation and validation

The numerical implementation is developed against a shared executable specification. OpenMM and the browser kernels receive the same molecular graph, atom ordering, coordinates, parameters, units, exceptions, constraints, and named energy terms. This enables term-resolved differential testing rather than comparison of independently prepared end states. The implementation loop is:

1. freeze the molecular graph, atom ordering, coordinates, force-field parameters, and units used by the reference calculation;
2. implement one energy/force term and its memory representation at a time;
3. evaluate the identical numerical input with OpenMM;
4. compare named energy components and Cartesian force components;
5. localize the first discrepancy, correct it, and retain the minimal reproducing case as a regression test; and

- execute the resulting WebGPU and WebAssembly paths in the browser to include worker, adapter, serialization, and UI integration boundaries.

Coding agents operate inside this harness by modifying implementation code, tests, and adapters. The physical-model equivalence criteria—including atom order, solvent model, cutoffs, constraints, units, charge provenance, and acceptable numerical tolerances—are specified explicitly and remain prerequisites for interpreting any backend comparison.

Several observed failures led directly to architectural changes (Table 3).

Table 3: Observed failures that changed the implementation.

Observation	Design change	Retained check
Pair metadata grew as $32N^2$	Store only exceptional pairs; mix ordinary pairs on demand	Large-system allocation and parity tests
32-bit force/energy sums overflowed	Paired-word 64-bit fixed-point accumulation and fault flags	Clashed system at approximately 2 million kcal/mol
Coordinates wrapped beyond 128 Å	64-bit fixed-point coordinate store plus floating-point working copy	Whole-system translation to 500 Å
Standard cutoff reduced performance	Retain for validation; omit from product controls	Fixed Trp-cage/ubiquitin timing protocols
Apparent engine mismatch	Verify atom order, input hashes, and solvent mode before numerical debugging	Native/WebAssembly parity and matched-solvent comparisons

Regression coverage is organized around the interfaces where failures were observed: equation-level fixtures for bonded and nonbonded terms; representation tests for clashes, constraints, overflow, and translated coordinates; backend comparisons on identical prepared inputs; protein fixtures that exercise special pairs and worker transfer; browser tests for adapter support and worker recovery; and fixed performance protocols that retain slower but numerically valid designs. Fixed random seeds and stable atom identifiers make failing cases repeatable. The replica engine terminates on fixed-point overflow or unconverged constraints; the direct engine records final constraint error for comparison against an acceptance threshold.

The demonstrated portability boundary is fixed-topology, nonperiodic molecular mechanics with regular array state and a small number of synchronization points. WebGPU is effective when work decomposes into local or pairwise operations and repeated evaluations amortize dispatch. Algorithms requiring 64-bit floating-point arithmetic, frequent device-wide synchronization, rapidly changing graphs, large adaptive sparse structures, or mature FFT/sparse-linear-algebra libraries require a different decomposition under current WGSL [22]. Particle-mesh Ewald electrostatics, heterogeneous reactive systems, and some long-timescale production protocols are therefore outside the demonstrated browser-native boundary. For those workloads, the architecture supports escalation to a WebAssembly reference, hybrid local path, or external specialist engine while preserving the originating molecular state and protocol provenance.

A.3 Workflow orchestration and local execution boundary

Molarium distinguishes local calculation from offline operation. In normal connected mode, supported numerical calculations execute on the user’s device, while explicit PDB/CCD retrieval or sequence-alignment searches contact the services identified in the interface. Local Lab serves a reviewed, versioned checkout from localhost under a restrictive Content Security Policy. External structure and alignment controls are disabled, and an egress-canary test fails if a synthetic private-data marker reaches an external request. A hosted application can change the assets served on a subsequent visit; the stronger reproducibility and privacy boundary is a reviewed checkout with locally pinned assets.

Current workflow surfaces include:

- **Local small-molecule design.** Synchronized 2D/3D construction, graph validation, RDKit or direct-WebGPU calculations, aligned trajectory/energy inspection, and export. OpenMM/WebAssembly is used for internal validation and fixed-scaffold operations rather than as a selectable Run-panel method.
- **Protein–ligand pose propagation.** A trusted pose is captured with stable atom lineage; changed ligand

branches are sampled while the receptor remains fixed; optional contact hypotheses constrain search; and a selected pose can be applied and relaxed. An experimental induced-fit action releases the ligand and all atoms of protein residues within a 6 Å pocket shell while outer atoms remain fixed.

- **Agent-operated design.** Agents invoke the versioned Chemist Actions API rather than arbitrary page functions. The API exposes the same bounded state-changing verbs available to a chemist—including select, edit, search, apply, relax, and undo—plus read-only inspection. Requests execute serially and record sequence, timestamps, arguments, status, and duration.

Two extension paths are represented in the architecture but are not complete product workflows. A local-to-specialist path can export a fingerprinted checkpoint and explicit protocol for periodic solvent, long trajectories, free energy, or electronic-structure calculations, with returned results attached as evidence to the originating branch; transport, scheduling, signatures, and trust policy are not implemented. A collaborative review model can branch by chemotype, protonation state, pose, or author and record a selected branch with explicit parentage; coordinate averaging is not used as a merge operation.

A.4 Content-addressed molecular and decision provenance

Molarium separates operational, numerical, and decision provenance. A molecular snapshot records one explicit state. Content-addressed commits link that state to parent states, action scripts, hypotheses, and evidence. Branches preserve alternative design routes, and decisions record which route progressed, stopped, or was deferred. These semantics are implemented as a custom content-addressed ledger rather than a Git repository or a general structural merge algorithm.

The current live application retains at most 500 action records in memory; clearing the scene clears that audit. Durable provenance therefore requires explicit export of API moves or a replay log. The append-only design-campaign ledger, commit, branch, decision, and verification functions exist in the repository and support standalone builders and reviews, while one-click promotion of a live session into ledger commits is not currently exposed in the main interface.

Table 4: Separation of operational, numerical, and decision provenance.

Record	Question	Contents
Chemist Actions audit	What was requested?	Public action, arguments, timing, status, and compact after-state
Calculation record	What numerical protocol ran?	Inputs, method, parameters, outputs, and hash-linked events
Design campaign	Why did a state advance or stop?	Snapshots, branches, parents, hypotheses, evidence, and decisions

Canonical JSON and SHA-256 fingerprints bind records to exact serialized representations. The `molarium.design-campaign/v1` schema stores snapshots, commits, branches, events, and decisions. Each commit identifies its selected snapshot, parent commit(s), optional action script, hypotheses, evidence, tags, and branch. Events are append-only and contain the previous event fingerprint, actor class (human, agent, system, or import), and source. Supported decision states are **progressed**, **not-progressed**, **deferred**, **failed**, **duplicate**, **superseded**, and **archived**. Finalization appends a completion event and hashes the campaign; subsequent record changes fail verification. Hashes provide alteration detection for the recorded representation, not authorship, chemical equivalence, or scientific validity.

Registered design routes use a separate `molarium.registered-design-route/v1` schema that specifies the coordinate boundary, hash-pinned starting state, and ordered graph edits. The SOS1, BCL-xL, and CDK2 routes use this schema. Route actions are `designRoute.load`, `designRoute.applyStep`, and `designRoute.inspect`; campaign-ledger operations remain separate. Cross-schema validation rejects a route presented as a campaign ledger or a campaign ledger presented as a route. Frozen predictions made before the schema split retain their original input hash, with migration records storing both hashes and accepting only the schema-identifier substitution that reconstructs the frozen bytes.

A decision-to-state linkage has the following form; full identifiers are SHA-256-derived values rather than the abbreviated labels shown here.

```

{
  "moleculeCommit": {
    "snapshotId": "snapshot:<selected-child>",
    "parents": ["commit:<parent>"],
    "actionScriptId": "script:<designer-moves>",
    "hypothesisIds": ["event:<pocket-must-open>"]
  },
  "decisionEvent": {
    "kind": "decision.recorded",
    "actorId": "chemist.01",
    "payload": {
      "targetCommitId": "commit:<selected-child>",
      "disposition": "progressed",
      "reasonCodes": ["contact-loss"],
      "rationale": "Phe-out removes growth-induced clashes.",
      "evidenceIds": ["event:<rotamer-search>"]
    }
  }
}

```

Executable replays use `molarium.chemist-action-script/v1`. Scripts contain ordered calls to the allow-listed Chemist Actions API rather than arbitrary code or direct coordinate replacement. Imports are capped at 2 MiB; one action’s arguments are capped at 32 KiB, eight nested levels, and 2,048 values. Actions can name returned stable identifiers for reuse by later steps. The runner executes one action at a time through `api.execute({action,args})`, derives a repeatable request ID from the script fingerprint, and stops on the first failure. Replay results use `molarium.chemist-action-replay/v1` and record the source fingerprint and each step outcome. Presentation actions can alter focus or styling but cannot change molecular graph or coordinates except through the same allow-listed state-changing actions.

The SOS1 replay is identified by a stable public route and by exact release and content fingerprints. A human-readable URL can resolve to a newer release, whereas the release commit, route-file fingerprint, and canonical action-script fingerprint identify the reviewed bytes and normalized JSON content. The live workflow can capture a starting state, record exploration actions, apply and relax selected states, and export action and numerical records. Formal campaign commits and decision events are represented by the ledger layer described above.

A.5 SOS1 hit-to-BAY-293 executable replay

The SOS1 example follows the analogue series reported by Hillig and colleagues [30]. The supplied design information consisted of the 5OVE/AXE coordinate-bearing starting complex and the published analogue graphs for AWT, AWZ, AWW, and AXH. The later analogue poses and receptor conformations were withheld during prediction. Each analogue was propagated from the preceding frozen prediction rather than from its corresponding later crystal structure.

The selected route contains 33 recorded API actions: 29 molecular or protocol operations and four workspace switches. The full exploration audit contains 89 completed API calls, including three Phe890/pose branches, nine undo calls, and deterministic reselection. The rendered interface sequence adds 15 display-only actions for 48 replay steps. Later crystal structures were opened only after four prediction checkpoints and the exploration audit had been hashed. Display and focus actions are non-mutating.

The analogue transitions involve substantial graph rewrites while preserving explicit atom lineage (Table 5).

Table 5: Heavy-atom lineage across the supplied SOS1 analogue graphs.

Transition	Retained	Deleted	Added	Candidate mappings
AXE-AWT	23	4	6	4
AWT-AWZ	19	10	12	1
AWZ-AWW	16	15	15	1
AWW-AXH	15	16	17	1

Each “64-chain” search comprises 64 independently seeded local pose searches. Parameterization without motion assigns the numerical force-field representation while preserving coordinates; only the subsequent relaxation changes coordinates. The SOS1 route uses whole-system Sage 2.1.0/Gasteiger preparation rather than the prepared Rosemary validation fixtures. AWT, AWZ, AWW, and AXH contain 7,929, 7,933, 7,935, and 7,937 particles, respectively. This charge assignment is part of the recorded protocol and is distinct from standard Sage workflows using AM1-BCC or NAGL charges.

Before flexible relaxation, Molarium stores ligand bond lengths and rejects a relaxed pose if any heavy-atom bond is both more than 0.45 Å longer and 1.25-fold its assigned target, or both more than 0.35 Å shorter and below 0.75-fold its target. Rejected relaxation restores the previous pose rather than accepting an improved score with a broken molecular geometry.

Table 6: Sequence of molecular states in the SOS1 replay. Predictions, rather than future crystal structures, seed the next step.

Step	Operation	State/evidence
0	Load and prepare	5OVE/AXE only; capture the compound 1 reference and coordinate boundary
1	Rewrite to compound 17 (AWT)	Run 64 independent reference-guided pose searches; apply selected pose; parameterize; relax
2	Rewrite to compound 18 (AWZ)	Propagate from predicted AWT, not 5OVF; apply, parameterize, relax
3	Grow compound 21 (AWW)	New ring enters the Phe890-in volume; fixed-core representative gains four clashes
4	Resolve pocket state	Enumerate discrete Phe890 rotamers; select outward branch; retain the moved receptor as the next reference
5	Predict BAY-293 (AXH)	Grow from frozen AWW; 64-chain pose search; apply; parameterize without motion; relax; freeze
6	Evaluate holdouts	Compare AWT/AWZ/AWW/BAY-293 with 5OVF/5OVG/5OVH/5OVI

The Phe890 conformational step is treated as a discrete state search because local minimization does not reliably cross the relevant side-chain torsional barrier. Separate rotamer basins are generated before local ligand–pocket relaxation. Feasible branches are ranked first by clashes introduced by ligand growth, then by pose score and finite relaxed energy. The nominal -60° branch adds four clashes above the fixed-core baseline and is rejected; the -180° branch adds none and is selected; the $+60^\circ$ branch adds two and remains feasible. The outward Phe890 χ_1 basin is therefore selected before 5OVH or 5OVI is revealed.

For post-freeze structural evaluation, predicted receptors are aligned to their holdouts using 23, 25, 25, and 27 pocket C α atoms for AWT through AXH. Ligand RMSD is then calculated over 29, 31, 31, and 32 heavy atoms, minimizing over 2, 1, 1, and 1 graph-symmetry mappings, respectively; the ligand itself is not fitted. Results are reported in Table 7.

Table 7: Post-freeze structural evaluation. Torsions are predicted/crystal in degrees; ligand RMSD is receptor-aligned and graph-symmetry-minimized.

Prediction	Holdout	Ligand RMSD (Å)	Phe890 χ_1	Phe890 χ_2
AWT	5OVF	0.516	-77.6 / -71.5	-71.7 / -71.7
AWZ	5OVG	1.528	-73.4 / -75.6	-92.0 / -87.9
AWW	5OVH	2.137	-172.2 / +175.8	+116.1 / +76.3
BAY-293	5OVI	1.215	-172.4 / -166.5	+108.7 / +71.7

The replay recovers the experimentally observed outward Phe890 χ_1 basin before exposure to 5OVH or 5OVI. The terminal χ_2 orientation is less accurate, differing by approximately 40° for AWW and 37° for BAY-293. The corresponding ligand RMSDs range from 0.516 to 2.137 Å under receptor-aligned, symmetry-minimized evaluation.

The interface replay executes from a blank canvas through the public Chemist Actions API. Candidate-generation operations leave coordinates unchanged until an explicit Apply action. The replay therefore represents discrete state

transitions rather than interpolated trajectories or molecular dynamics. Its 48-action rendered sequence lasts 62.58 s while the underlying audit records approximately 908.5 s of operation durations; timing remains attached to the audit and render manifest, and the rendered sequence is not used as a performance benchmark. Comparison of fragment-bound 6EPM with the 5OVE starting structure shows that the inhibitory Tyr884 arrangement is already present at the coordinate boundary; the hit-only route does not predict a Tyr884 transition.

APPENDIX B

Molarium operating manual and implementation reference

This appendix is the operational companion to Appendix A. It records the inspected implementation surface, entry points, state mutations, persisted records, failure behaviour, and explicit boundaries of individual Molarium modules. The reference was inspected on 2 September 2026 on the feature branch through commit `ae052c4`. The status categories below distinguish current browser behaviour, experimental but executable behaviour, repository tooling, proposed functionality, and retired or compatibility-only code. These labels are part of the scientific contract: the presence of code in a repository does not by itself imply that a capability is supported, validated, or exposed to end users.

B.1 Scope and status conventions

This manual describes `feature/sos1-interface-story` through commit `ae052c4` on 2026-09-02. The blind implementation/history pass covered the repository through `663e4a71578d`; subsequent factual review incorporates the route/ledger schema split and the pre-release `designRoute.*` API cleanup. The manual is derived from implementation, tests, module READMEs, and Git history. It does not describe intent that has no executable implementation. Status terms used below:

- Current browser surface - reachable from the main application UI or its installed browser API. The source entry points are `index.html`, `app.js`, and `chemist-actions.mjs`; these files were byte-identical to their counterparts in `dist/` at inspection time.
- Experimental current surface - executable from the current browser application, but explicitly labeled experimental or subject to material constraints in the implementation or module README.
- Repository tooling - executable tests, command-line utilities, registries, and standalone viewers that are not general controls in the main workbench.
- Proposed - design text or a preregistered/development record without a corresponding general runtime path. Proposed items are not operating instructions.
- Retired/compatibility-only - code or assets preserved for compatibility or history but not presented as a current workbench choice. The application is a single-page browser workbench. Its default launch loads a bundled LSD molfile; `?blank` starts with no molecule. A registered story URL instead verifies and installs an action script on a blank canvas (`app.js:13938-14008`).

B.2 Runtime, distribution, and network policy

Responsibility. Serve the static browser application, select connected or Local Lab behavior, and verify large runtime assets before a worker uses them.

Entry points and controls.

- `bun run start` (or `bun server.js`) starts the Bun server; `bun run start:local` selects Local Lab. `-port`, `PORT`, `-local-only`, and `-test-api` alter the server mode (`server.js:1-105`; `package.json:6-16`).
- A static host uses the checked-in `runtime-config.js`, whose defaults are connected mode with the test API disabled (`runtime-config.js:1-11`).
- The project information dialog exposes the current network policy and a build-verification action (`app.js:13892-13919`).

Inputs, outputs, and state.

- Input: HTTP requests for application files and, in connected mode, external structure, component-dictionary, and MSA requests.
- Output: static HTML, JavaScript, styles, model/runtime assets, and a small runtime configuration object.
- Mutation: server mode changes which features are enabled in the rendered session. It does not create a server-side project record.

- External boundary: connected mode may contact RCSB for PDB/CCD data and a configured MSA service. Local Lab permits same-origin resources only and applies restrictive response headers (`server.js:11-44`, `server.js:73-85`; `README.md:2560`).
- Persisted records: deployment builds copy the application to dist/; large assets use versioned object paths and hash manifests (`DEPLOYMENT.md:1-80`).

Failure behavior and limits.

- Asset fetch, size, or SHA-256 mismatch rejects the worker initialization; a rejected verification promise is cleared so a later attempt can retry (`app.js:10033-10155`).
- A normalized request path that does not resolve to a file returns an HTTP error rather than arbitrary filesystem content (`server.js:46-105`).
- Local Lab disables controls requiring external PDB, CCD, or MSA access. This is an operating mode, not an offline cache

manager.

- The repository describes Cloudflare Pages plus versioned object storage, but the checked-in source cannot establish which commit is live at an external URL. "Current" in this manual therefore means the inspected build, not a claim about an independently hosted instance.
- `-test-api` is a privileged development mode. Production intentionally exposes Chemist Actions but not the browser-test harness (`browser-test.js:122-133`; `app.js:9701-9703`).

B.3 Workspace shell and session state

Responsibility. Own the current molecular graph, visual view, edit transaction, undo/redo stacks, calculation results, docking state, design replay state, component visibility, and lazy worker handles.

Entry points and controls.

- Main modes: View, Build, and Run (`index.html:529-531`). Structure loading remains available in the left panel.
- Viewer toolbar: Export, Clear, Share, Save, Finish chemistry, Discard, and Fullscreen (`index.html:630-637`).
- Public automation: `window.MolariumChemistActionsReady`, which resolves to the installed Chemist Actions API (`app.js:14005-14007`).

Inputs, outputs, and state mutations.

- The central in-memory state object contains the molecule, camera, selections, pending chemistry, build/redo histories, calculation frames and ensembles, docking reference/results, registered design route, designer script/replay, component visibility, depiction state, and worker maps (`app.js:655-766`).
- Loading a molecule stops calculation playback, resets incompatible calculation, docking, selection, and preparation state, and chooses an initial representation appropriate to the source (`app.js:3078-3142`).
- Clear removes the molecule and all associated edit, calculation, docking, registered-route, audit, component, focus, and depiction state (`app.js:13838-13866`).

Persisted/exported records.

- There is no implemented local-storage, IndexedDB, server project, or automatic session restore path.
- Share copies the current URL, or a registered story permalink. It does not serialize the current molecule or session (`app.js:13869-13875`).
- Save writes a PNG of the 3D canvas (`app.js:13877-13881`). Other exports are module-specific and listed in section 18.

Failure behavior and limits.

- Reloading or clearing the page discards unsaved session state.

- Worker instances are lazy and session-scoped. Worker errors reject outstanding work, terminate the failed worker, and remove it so a later request can recreate it (`app.js:10167-10205`).
- The workbench does not provide a general project bundle containing structure, view, edit history, calculations, and docking records.

B.4 Structure ingestion

Responsibility. Create the current molecular graph from pasted coordinates, uploaded text, an identifier, a bundled example, or a registered story.

Entry points and controls.

- Paste coordinates, Upload file, or enter an identifier (`index.html:73-115`).
- Identifier input accepts the built-in example names, a PDB identifier, or a SMILES string (`app.js:13422-13460`).
- Prepared examples are a separate selector (`index.html:178-183`).
- The launch asset is `assets/lsd-launch.mol`; failure falls back to an embedded XYZ example (`app.js:13938-13954`).

Inputs and outputs.

- XYZ: optional atom-count and comment lines, then element and three coordinates. Invalid coordinate lines are ignored; at least one valid atom is required. Coordinates are centered, and bonds are inferred from covalent radii (`app.js:1204-1228`).
- MOL: the internal parser accepts V2000 atoms, bonds, and formal-charge records, then centers the result (`app.js:1231-1278`).
- PDB: only the first model is loaded. Alternate locations prefer blank or A, otherwise the highest occupancy. Protein templates, SSBOND, and CONECT records contribute bonds; hydrogens are assigned to the nearest same-residue partner (`app.js:1346-1458`).
- PDB identifiers are fetched from RCSB in connected mode and must be four characters with an initial digit (`app.js:13422-13431`).
- SMILES is embedded through the RDKit worker. Successful ingestion produces a molecule object and may immediately enumerate protonation states (`app.js:13434-13460`).

State mutations, boundaries, and records.

- Successful input replaces the current molecule and resets dependent state via `loadMolecule()` (`app.js:3078-3142`).
- External boundary: PDB identifier retrieval uses RCSB; SMILES embedding uses the local RDKit worker and its verified assets.
- The input structure itself is not persisted automatically. Source/provenance fields remain attached to the in-memory molecule.

Failure behavior and verified limits.

- Parse and fetch failures are shown as notices and leave the previous scene in place for paste/upload/identifier handlers (`app.js:13408-13479`).
- The upload control advertises `.xyz,.pdb,.mol`, but its handler calls `parseStructureInput()`, which routes only PDB versus XYZ. Uploaded MOL files therefore do not use the implemented V2000 parser (`index.html:94-99`; `app.js:1461-1465`; `app.js:13470-13479`). The bundled launch MOL is parsed through a separate explicit path.
- The MOL parser is V2000-only. The PDB loader is not a trajectory or multi-model reader.
- XYZ bond inference is proximity-based. Interactive Builder chemistry, by contrast, operates on explicit graph bonds.

B.5 Ligand protonation

Responsibility. Enumerate and apply ligand protonation states at an operator-selected pH.

Entry points and controls.

- The Load panel exposes pH from 0 to 14, state selection, and Apply ([index.html:116–131](#)).
- The same functions are available through Chemist Actions under the protein/structure routes installed in the application ([app.js:8243–8711](#)).

Inputs, outputs, and mutations.

- Input: current ligand SMILES and pH.
- RDKit matches a versioned Dimorphite-style site table, edits formal charges and hydrogens, sanitizes results, and reports an ordered state list with weights ([rdkit-worker.js:174–204](#); [rdkit/README.md:23–32](#)).
- Apply re-embeds the chosen state, replaces the current molecular graph and coordinates, and invalidates state that depends on the prior topology.

Boundary, records, failures, and limits.

- Boundary: dedicated RDKit Web Worker using pinned local runtime assets.
- Record: selected state and enumeration information exist in session state; no standalone protonation-record export is implemented.
- Errors from invalid pH, sanitization, embedding, or worker startup are surfaced and do not partially apply a state.
- Reported weights are independent Henderson-Hasselbalch estimates. They are not microscopic-pKa or coupled-site populations ([rdkit/README.md:23–32](#)).

B.6 PDB preparation and parameterization

Responsibility. Inspect a loaded PDB, apply a bounded set of repairs and protonation choices, and, if no blocker remains, create a complete numeric simulation system.

Entry points and controls.

- Preparation controls select pH, histidine policy, heavy-atom repair, ligand policy, water policy, and gap policy ([index.html:133–177](#)).
- The right-side preparation inspector reports issues and downloads a preparation report ([index.html:686–698](#)).

Inputs and outputs.

- Options normalize to: histidine auto/hid/hie/hip; repair missing supported heavy atoms; ligand ccd/exclude; water crucial/retain/exclude; and gap cap/block ([app.js:2884–2905](#)).
- Preview inventories residues, missing atoms, gaps, ligands, water, clashes, and invalid coordinates. It may retrieve CCD data, reconstruct supported missing heavy atoms, add standard hydrogens and terminal oxygen, select histidine states, and scan 24 polar-hydrogen orientations ([app.js:2946–3056](#)).
- Apply refuses a preview with blockers, runs the OpenMM worker’s parameterization path, replaces the molecule with the prepared version, marks it parameterized-experimental, and adds an undo snapshot ([app.js:9965–10021](#)).

State mutations, boundaries, and records.

- Preview updates `state.pdbPreparationPreview` without committing coordinates.
- Apply mutates the molecular graph, coordinates, preparation audit, and parameterization system.
- External boundary: CCD lookup in connected mode. Compute boundary: OpenMM worker and parameterization modules.
- Export: preparation-report JSON ([app.js:3059–3064](#)). The molecule also carries its preparation audit and parameterization in memory.

Failure behavior and verified limits.

- Unsupported residues, unsupported missing atoms, incomplete retained ligands, blocked gaps, invalid coordinates, or severe clashes are blockers. Apply reports the blocker rather than manufacturing a production claim.
- The preview explicitly records `readyForProductionSimulation: false` (`app.js:3050`).
- Current preparation does not reconstruct missing loops, build membranes, model metal coordination, parameterize covalent ligands, or create periodic solvent boxes (`README.md:305–343`).
- Arbitrary proteins use an experimental Sage/Gasteiger route. Bundled prepared Rosemary fixtures are exact preparameterized systems and do not demonstrate general browser-native protein typing (`openff/README.md:23–58`; `README.md:305–343`).

B.7 Molecular viewer, components, and synchronized depiction

Responsibility. Display the same molecular graph in interactive 3D and, for a selected small-molecule component, RDKit-generated 2D; provide atom, residue, component, trajectory-frame, and replica selection.

Entry points and controls.

- Pointer drag rotates; Shift-drag pans; wheel zooms. Click selects atoms. Viewer display controls select representation, color mode, labels, axes, auto-rotation, and measurement (`app.js:13728–13835`; `index.html:214–247`).
- Component controls show/hide and focus connected components (`index.html:671–683`; `app.js:2342–2412`).
- Protein controls expose pocket, contact, residue-focus, and interaction views.
- Result controls select saved frames, conformers, and STORMM replicas (`index.html:730–790`; `app.js:12398–12476`).

Inputs, outputs, and mutations.

- Input: current graph, coordinates, residue annotations, component assignments, display settings, and optional frame/replica arrays.
- Output: canvas rendering, scene preview, component list, selection details, noncovalent-contact overlays, and sanitized RDKit SVG depiction.
- State mutations are view-only unless the operator invokes a Builder action. Camera, focus, visibility, selected atom(s), selected residue, frame, and replica are session state (`app.js:655–766`).

Boundary, records, failures, and limits.

- The 2D depiction boundary is the RDKit worker. The 3D viewer is application canvas code.
- Export: PNG captures the 3D canvas. View settings are not included in a general session export.
- The 2D target is one component and is capped at 256 atoms (`rdkit-worker.js:207–248`). A failed depiction leaves the 3D molecule usable and shows an unavailable depiction state.
- Protein covalent atoms are protected from ordinary ligand chemistry edits. Local cleanup operates on a bounded neighborhood (`README.md:176–225`).
- Trajectory alignment is for display only; raw calculation coordinates are retained for replica/frame data and coordinate export (`app.js:11340–11662`; `README.md:176–225`).
- Pocket membership is a contact-oriented cutoff view, not a binding-site prediction. The default pocket/contact construction uses the documented 5 Å neighborhood (`README.md:176–225`).

B.8 Builder and chemistry transactions

Responsibility. Modify graph topology, atom properties, stereochemistry, local geometry, fragments, and selected coordinates while maintaining an undoable molecular state.

Entry points and controls.

- Undo/Redo; fragment staging and templates; element replacement; geometry controls; bond order and aromaticity; formal charge; explicit hydrogen; stereochemistry; sidechain rotamers; designer moves; and optimization controls are in the Build panel (`index.html:249–408`, `index.html:701–723`).

- Geometry selection uses connected paths of two, three, or four atoms for distance, angle, or dihedral operations (`app.js:6354-6540`).

Inputs, outputs, and state mutations.

- A chemistry-changing action opens or updates one chemistryTransaction, captures the pre-edit molecule, applies graph mutations, reconciles hydrogens, maintains stable atom identifiers, and records changed atoms (`app.js:6541-6769`).
- Finish sanitizes through RDKit, performs local coordinate cleanup, remaps reference contacts when applicable, invalidates obsolete parameterization, and pushes one undo snapshot for the transaction (`app.js:6825-6894`).
- Discard restores the pre-transaction molecule (`app.js:6897-6906`).
- Coordinate-only geometry actions move the connected side selected by the graph traversal; ring restrictions prevent ambiguous cuts.

Boundaries and records.

- RDKit sanitization and eligible RDKit cleanup run in a worker. WebGPU and ANI optimization routes use their own workers.
- The in-memory build/redo stacks retain snapshots. Chemist Actions adds a bounded audit record; design replay can turn completed public actions into an exported script.

Failure behavior and verified limits.

- A synchronous mutation failure restores the transaction snapshot. A Finish failure keeps the transaction pending so the operator can continue editing or discard it (`app.js:6695-6769`; `app.js:6825-6906`).
- If the final graph is valid but local coordinate polish fails, the valid chemistry remains committed and the error is stored in source metadata (`app.js:6783-6822`).
- Fused-aromatic bond editing is rejected where a local bond-order change cannot preserve the aromatic graph (`app.js:6282-6311`).
- RDKit optimization is disabled for parameterized protein complexes; pocket methods require a complex; ANI availability is separately gated (`app.js:12880-13015`).
- Optimization captures ligand valence geometry and restores the pre-optimization coordinates if the result crosses the implemented safeguard (`app.js:13139-13227`; `DESIGNER-MOVES.md:158-162`).
- The induced-fit control is experimental: it moves the ligand plus complete residues within 6 Å while fixing the outer system. It is not an affinity or ensemble calculation (`DESIGNER-MOVES.md:87-90`).

B.9 RDKit subsystem

Responsibility. Provide SMILES embedding, sanitization, 2D depiction, protonation, small-molecule conformer seeds, force-field energy, local geometry optimization, and short dynamics.

Entry points.

- Main UI: identifier/SMILES load, 2D view, Finish chemistry, ligand optimization, RDKit Run method, and STORMM conformer seeding.
- Worker jobs: smiles-depict, depict, protonation, smiles-embed, sanitize, conformers, energy, geometry, and dynamics (`rdkitworker.js:174-469`).

Inputs, outputs, state, and boundary.

- Input: molecular graph or SMILES, coordinates, fixed-atom set where supported, and job settings.
- Output: sanitized graph/SMILES, SVG, embedded coordinates, conformer arrays, energies, optimized coordinates, or saved dynamics frames.
- Mutations occur only when the application applies the returned graph or coordinates.
- Boundary: dedicated Web Worker and pinned RDKit WebAssembly assets with recorded hashes (`rdkit/README.md:34-51`).

Records, failure behavior, and limits.

- Calculation results store RDKit version, force field/fallback, timing, energies, and frames in session (`app.js:12620-12653`).
- Worker errors include the job identifier and reject the pending application request.
- Fixed atoms are restored exactly after the V2000 round trip used for optimization (`rdkit-worker.js:419-429`).
- Conformer count is capped at 256. Large requests can use up to four RDKit workers for seed generation (`rdkit-worker.js:250-469`; `README.md:99-101`).
- Depiction is capped at 256 atoms for the selected component.
- The RDKit dynamics path reports a 0.1 fs timestep and is a short workbench calculation, not a production trajectory generator (`rdkit-worker.js:430-469`).

B.10 OpenFF/Sage parameterization

Responsibility. Convert a supported molecular graph into the numeric bonded, nonbonded, charge, constraint, and optional implicit-solvent arrays consumed by calculation workers.

Entry points.

- Automatic prerequisite to direct WebGPU, STORMM on the current molecule, docking, and preparation workflows.
- There is no separate general "parameterize" button for an arbitrary ligand.

Inputs, outputs, state, and boundary.

- Input: atoms, bonds, coordinates, charge, and selected simulation options.
- The parameterizer converts to a mol block, checks supported elements, loads the Sage schema, uses RDKit for SMIRKS matching and deterministic Gasteiger charges, and requires every needed parameter assignment (`openff/sage-parameterizer.js:22-339`).
- Output: a complete numeric system plus parameter labels, force-field identity, charge model, and source hash.
- Successful automatic parameterization is cached on molecule.parameterization (`app.js:12563-12577`). Topology-changing edits invalidate it.
- Boundary: OpenFF JavaScript modules plus RDKit worker services.

Records, failure behavior, and limits.

- Parameterization metadata is shown with calculation results and retained in the current molecule.
- Unsupported elements, invalid chemistry, or an unmatched required parameter fail the request; workers do not silently omit the term.
- Charge assignment is deterministic Gasteiger, not AM1-BCC or NAGL (`openff/README.md:1-21`).
- Simulation options implement no constraints or X-H constraints, nonperiodic cutoff validation, timestep validation, and OBC2/ACE implicit solvent (`openff/simulation-options.js:14-87`; `openff/implicit-solvent.js:14-59`).

B.11 Direct WebGPU force-field engine

Responsibility. Evaluate the numeric Sage system directly on WebGPU for energy, geometry optimization, and dynamics.

Status and entry points. Experimental current surface. Select Direct WebGPU in Run, or use WebGPU optimization controls in Build (`index.html:412-520`; `app.js:12707-12719`).

Inputs, outputs, state, and boundary.

- Input: parameterized system, coordinates, job type, step count, solvent/constraint options, saved-frame count, and optional movable atoms for geometry optimization.

- Output: positions, forces, total and component energies, convergence/timing metadata, and frames for dynamics (`webgpuworker.js:590-699`).
- Geometry/dynamics positions replace the current coordinates; energy does not. Calculation frames and metadata populate the result card (`app.js:12552-12724`).
- Boundary: browser GPU adapter/device and verified shader/runtime assets. Buffers are destroyed after the job.

Records, failure behavior, and limits.

- In-session record includes force field, charge model, solvent, constraints, cutoff, active/fixed atom counts, backend, elapsed time, energies, and frames.
- Missing WebGPU, insufficient adapter limits, invalid/non-finite data, or worker/device errors fail the calculation and populate `lastCalculation.error` (`app.js:12727-12738`).
- The application caps direct dynamics at 5,000 steps (`app.js:12508-12530`).
- The engine uses f32 and has no periodic boundaries, PME, virtual sites, barostat, explicit-solvent setup, generic protein parameterization, or batched independent-system execution. A cutoff path exists in the numeric layer, but normal workbench workflows use the displayed nonperiodic/no-cutoff configuration (`webgpu/README.md:1-48`; `README.md:154-174`).

B.12 OpenMM worker

Responsibility. Run the complete numeric system with OpenMM 8.2 Reference in browser WebAssembly for internal parameterization, scoring, fixed-scaffold relaxation, validation, and compatibility workflows.

Status and entry points. Internal current subsystem. `runCalculation()` accepts an openmm override, but the Run method selector does not expose OpenMM; visible choices are Direct WebGPU, WebGPU ensemble, RDKit, and ANI-2x (`app.js:12508-12555`; `index.html:433-438`).

Inputs, outputs, state, and boundary.

- Worker jobs include parameters, energy, geometry, dynamics, conformer benchmark/fixed-conformer work, and score (`openmmworker.js:253-568`).
- Input is a molecule or complete numeric system. Output is parameters, energy, coordinates, frames, and platform/version metadata.
- Boundary: OpenMM 8.2 WebAssembly using the double-precision Reference platform (`openmm/README.md:1-15`).

Records, failure behavior, and limits.

- Internal callers retain only the records their parent workflow stores, such as preparation audits, docking refinements, or validation results.
- Initialization, context construction, parameter, or non-finite-result errors reject the calling workflow.
- Execution is nonperiodic. The older MolariumFF entry remains in the binary for compatibility but is not exposed in the current UI (`openmm/README.md:1-15`).

B.13 STORMM WebGPU ensemble

Responsibility. Run many homogeneous replicas of one topology for molecular dynamics or the anneal/minimize conformer protocol.

Status and entry points. Experimental current surface. Choose WebGPU ensemble with Dynamics or Conformer search. Controls select current/prepared system, replica count, conformer count/effort/cluster cutoff, comparison arena, and saved frames (`index.html:412-520`).

Inputs, outputs, state, and boundary.

- Input: one numeric topology, initial coordinates or RDKit conformer seeds, 1-1,024 replicas, dynamics/conformer settings, constraints, and solvent.
- Conformer protocol uses deterministic seeded stages: initial relaxation, 600 K, 450 K, 300 K, and final relaxation (openff/ `conformer-protocol.js`:5-29).
- Output: replica-major coordinates/frames, energies, constraint diagnostics, timing, and conformer analysis (stormmm-worker. `js`:187-377).
- The selected replica or conformer updates displayed/current coordinates. Raw replica frames remain separate from display alignment (`app.js`:12398-12476).
- Boundary: WebGPU engine with deterministic integer accumulation/storage and an f32 coordinate mirror (stormmm/README. `md`:1-156).

Records, failure behavior, and limits.

- The result card stores replica/conformer counts, system identity, homogeneity, timing, frames, cluster analysis, and optional arena results (`app.js`:12620-12724).
- Worker validation rejects unsupported job types, replica counts, step counts, atom counts, excessive pair work, NaN values, and fixed-point overflow/clamp conditions (stormmm-worker. `js`:187-377).
- Limits include one homogeneous topology, at most 512 atoms per replica, at most 1,024 replicas and 100,000 steps per request, all-pairs nonperiodic interactions, no PME/PBC, and a bounded pair-work allocation (stormmm/README. `md`:142-156).
- Fixed-point spatial precision degrades when coordinates are far from the working origin; recenter structures before relying on tight positional comparisons (stormmm/README. `md`:142-156).

B.14 ANI-2x MLIP

Responsibility. Compute ANI-2x ensemble electronic energies and analytical forces for eligible small neutral molecules, with optional geometry optimization.

Status and entry points. Experimental current surface. Choose ANI-2x with Energy or Geometry. It is also an eligible Builder optimization route when the molecule passes the gate (`app.js`:12523-12530; `app.js`:12880-13015).

Inputs, outputs, state, and boundary.

- Input must be a connected, neutral, singlet molecule with explicit hydrogens, at most 96 atoms, and only H, C, N, O, S, F, or Cl (`mlip/ani2x.js`:1-68).
- Output includes total electronic energy, ensemble standard deviation, forces, optimized coordinates where requested, model hash, evaluation count, backend, and timing (`mlip-worker.js`; `app.js`:12620-12719).
- Geometry results replace current coordinates; energy results do not.
- Boundary: eight ONNX models through ONNX Runtime Web, preferring WebGPU and falling back to WASM; a CPU descriptor path is available (`mlip/README.md`:1-13).

Records, failure behavior, and limits.

- Result metadata records model, level, model-source hash, backend, evaluation count, and initial/final ensemble spread.
- Eligibility failures are reported before execution. Model download/init or non-finite results fail the request.
- No ions, radicals, disconnected systems, implicit-hydrogen inputs, unsupported elements, or molecules above 96 atoms are accepted. The result is an ANI electronic-energy model, not a Sage potential energy or binding affinity.

B.15 OpenFold protein prediction

Responsibility. Predict coordinates for one protein sequence using a remote MSA service and local OpenFold ONNX inference.

Status and entry points. Experimental current surface in connected mode. The Fold Protein panel accepts a sequence, MSA endpoint, and recycle settings, and exposes Run and Cancel (`index.html:188-209`). Local Lab disables the external path.

Inputs, outputs, state, and boundary.

- Input: one amino-acid sequence using the standard residue alphabet or X, with maximum length 128. Feature buckets are 64 or 128 residues (`openfold/features.js:1-69`).
- The MSA client submits the sequence to the configured HTTP(S) endpoint, polls its ticket, validates/decompresses the returned archive, and reports rate-limit, maintenance, and timeout states (`openfold/msa-client.js`).
- The predictor loads the matching ONNX bucket, prefers WebGPU and falls back to WASM, and uses three recycles by default (`openfold/predictor.js:52-171`).
- Success loads the predicted structure as the current molecule. Export writes predicted/imported protein coordinates as PDB (`app.js:9805-9841`; `app.js:13298-13303`).

Records, failure behavior, and limits.

- Session state records progress and prediction metadata. No server-side job ledger is implemented in this repository.
- Cancel aborts the active MSA/prediction controller. Fetch, archive, feature, model, and inference failures report a notice without a partial prediction (`app.js:9805-9841`).
- The sequence is transmitted to the configured MSA endpoint. First use may fetch about 540 MB of model assets (`index.html:188-209`).
- Only one entity is handled; length is capped at 128; templates and Amber relaxation are not implemented (`README.md:355-366`).

B.16 Calculation orchestration and results

Responsibility. Validate a Run request, prepare its engine, dispatch one worker job, apply coordinates, and present energy, progress, performance, frames, replicas, or conformer summaries.

Entry points and controls.

- Job: Geometry optimization, Single-point energy, Molecular dynamics, or Conformer search.
- Method: Direct WebGPU, WebGPU ensemble, RDKit, or ANI-2x.
- Settings include steps, solvent, X-H constraints, ensemble/system controls, conformer protocol, comparison arena, and saved frames (`index.html:412-520`).

Inputs, outputs, and mutations.

- Dispatch validates molecule presence, pending chemistry, concurrency, and method/job compatibility. Conformers require STORMM; STORMM otherwise supports dynamics; ANI supports energy/geometry (`app.js:12508-12530`).
- STORMM on the current molecule automatically parameterizes if needed. Conformer search first obtains RDKit seeds, then runs STORMM or the comparison arena (`app.js:12552-12619`).
- Non-energy jobs apply returned coordinates. Results populate raw/display frames, optional ensembles, `state.lastCalculation`, and the result card (`app.js:12620-12724`).

Boundary and records.

- Boundary: one lazy worker per calculation family. Progress messages update the modal overlay.
- In-session records include initial/final energies, engine/model versions and hashes, force field, charge model, solvent, constraints, cutoff, convergence, platform/backend, elapsed time, timestep, frames, replicas, and conformer analysis (`app.js:12620-12653`).
- Export offers the current frame as XYZ and conformer ensembles as SDF where applicable (`app.js:9705-9768`).

Failure behavior and verified limits.

- A pending chemistry transaction blocks Run. A second calculation is refused while one is active (`app.js:12515-12520`).
- Failure marks the result state error, stores the message in `lastCalculation`, restores the enabled Run button, and hides the progress overlay (`app.js:12727-12738`).
- The general Run overlay has no calculation-cancel control. OpenFold has its own cancellation path.
- Saved-frame choices are bounded UI samples, not a full trajectory archive (`index.html:510-517`). The current workbench has no dedicated generic trajectory-file export.
- Conformer SDF export requires V2000-sized records and otherwise reports an error (`app.js:9718-9768`).

B.17 Reference-guided docking

Responsibility. Carry a prepared reference pose through a local ligand edit, generate feasible candidate poses, score/rank them in a rigid local receptor site, and record the workflow in a tamper-evident labbook.

Status and entry points. Experimental current surface. In Build, capture a reference, choose Pose propagation or Expert selected core, choose cleanup/contact options and candidate count, then run constrained docking (`index.html:264-307`). The current protocol snapshots are ConstraintDock 0.4.0 and Pose Propagation 0.9.0 (`docking/README.md:56-91`).

Submodules.

- `docking/protocol.mjs` - immutable protocol/settings snapshot and status.
- `docking/reference-core.mjs` - stable atom identity, reference capture, mapping, and selected-core validation. Expert cores require at least three connected, non-collinear heavy atoms (`docking/reference-core.mjs:47-80`).
- `docking/browser-adapter.mjs` - extracts ligand/receptor plans, reference contacts, and applies a compatible pose.
- `docking/feature-seeding.mjs` - deterministic edited-region torsion/feature seeds for pose propagation.
- `docking/contact-remap.mjs` - perceives reference features and proposes compatible replacements at an edit boundary.
- `docking/transformed-ring-region.mjs` and `docking/ring-conformer-generator.mjs` - identify released ring systems and enumerate bounded ring variants.
- `docking/constraints.mjs` and `docking/manual-hbond.mjs` - evaluate hard positional/core and selected hydrogen-bond constraints.
- `docking/restraint-biased-search.mjs` - first searches a restraint objective, then sends feasible poses to physical scoring.
- `docking/receptor-score.mjs` - evaluates local ligand-receptor Lennard-Jones and Coulomb cross terms.
- `docking/workflow.mjs` - aligns/snaps, relaxes, deduplicates, ranks, and returns candidate records.
- `docking/sidechain-rotamers.mjs` - enumerates a bounded set of canonical sidechain alternatives for explicit selection/application.
- `docking/labbook.mjs` - append-only event records with chained SHA-256 hashes and Markdown rendering.

Inputs, outputs, and mutations.

- Capture requires a prepared complex and stores the ligand plan, stable atom identifiers, selected/reference core, receptor atoms within 8 Å, selected H bonds, and provenance (`app.js:4474-4640`).
- Run refuses pending chemistry, a concurrent docking run, or unavailable referenced contacts. It verifies receptor/contact integrity, maps surviving ligand atoms, releases transformed rings, generates seeds, freshly parameterizes the edited ligand, relaxes/scores candidates, and updates progress while periodically yielding to the UI (`app.js:4642-5428`).
- Output is a ranked list of distinct candidate poses plus completion summary and labbook. Candidates remain separate from the molecule until Apply. Apply rechecks topology and receptor integrity before replacing coordinates (`app.js:5335-5530`).

Boundaries and records.

- Boundaries: RDKit seed generation, OpenFF parameterization, WebGPU/OpenMM relaxation and scoring paths, all coordinated in the browser.
- Exports: labbook JSON and human-readable Markdown/notes. The record includes input settings, hashes, mapping/contact decisions, feasibility rejections, scores, and selected pose. Coordinate-free input hashes are intentional (`docking/README.md:110–125`).

Failure behavior and verified limits.

- Invalid reference core, changed receptor topology/coordinates, unmapable required atoms, unavailable contact features, parameterization error, or no feasible candidate produces an explicit failed/negative result; it does not silently apply the leastbad pose.
- Default Pose propagation maps surviving reference atoms and seeds edited regions. Expert selected-core is a narrower operatorcontrolled mode. Automatic MCS alignment of an independently imported analogue is not implemented (`README.md:263–267`).
- The scorer uses a rigid 8 Å receptor site, relative dielectric 4 cross terms, and relative vacuum ligand strain. It omits general receptor relaxation, solvent displacement, macrocycle/fused-ring concerted search, entropy, and binding free energy. This is local analogue pose maintenance, not global docking (`README.md:269–276`).
- The separate induced-fit Builder operation is not part of the docking ranker.

B.18 Chemist Actions API

Responsibility. Expose a fixed JSON-only browser API for inspection and the same structure/view/edit/docking/optimization/operations available through application routes. Entry point and controls

- Await `window.MolariumChemistActionsReady`, then call `api.execute({ action, args })`; `api.inspect(args)`, `api.describe()`, and `api.history()` provide the read-only shortcuts and records (`chemist-actions.mjs:149–211`; `CHEMIST-ACTIONS-API.md:1–155`).
- Action families cover session inspection, view, build, protein, selections, chemistry, history, pose/sidechains, optimization, registered design routes, and structure stories. Registered routes use `designRoute.load`, `designRoute.applyStep`, and `designRoute.inspect`; `designRoute.load` takes a `routeId` (`chemist-actions.mjs:3–105`).

Inputs, outputs, state, and boundary.

- Inputs are plain JSON. Maximum nesting is 8, node count 2,048, encoded size 32,768 bytes; prototype-related keys are forbidden (`chemist-actions.mjs:107–136`).
- Installed routes call application functions, so mutations and safeguards are the same as UI operations (`app.js:8243–8711`).
- Calls are serialized through one promise queue. Recognized calls create running, then completed or failed, audit entries; the bounded history defaults to 500 entries (`chemist-actions.mjs:149–211`).
- Boundary: same-page JavaScript API. It does not grant arbitrary code execution, direct coordinate injection, callbacks, or a generic network function.

Persisted/exported records, failures, and limits

- Audit is session state and can be converted into a Designer action script. Completed public actions are replay candidates; failed actions remain audit evidence.
- Unknown actions fail before an audit entry because no installed route exists. A route failure records the failed call and propagates its error (`chemist-actions.mjs:149–211`).
- Inspection defaults to a ligand-oriented, coordinate-free summary; coordinates require an explicit request and are capped by the documented atom limit (`CHEMIST-ACTIONS-API.md:147–155`).
- The API is a stable command boundary, not a privileged test surface. Browser-test helpers require `–test-api` and are not part of production (`browser-test.js:122–133`).

B.19 Designer replay, registered routes, campaign ledgers, and standalone viewers

Responsibility. Capture completed public actions as replayable scripts; replay them deterministically through the public API; validate registered graph-edit routes; maintain separate append-only, content-addressed campaign

ledgers; and render selected records in standalone viewers.

Status and entry points.

- Current browser surface: Import script, Restart, Play/step story, and Export script controls in Build (`index.html:342–359`). Registered story URLs load a hash-verified bundled script on a blank canvas (`app.js:13957–14003`).
- Current API surface: registered-route actions under `designRoute.*`, plus structure-story actions (`chemist-actions.mjs:73–105`). There is no pre-release compatibility alias.
- Standalone repository surfaces: `design-history/viewer/` and `design-history/structure-viewer/` consume bundled records independently of the main workbench.

Inputs, outputs, and state mutations.

- Replay accepts a validated script containing only public actions and binding captures. Import clears the existing molecule before installing the script (`design-history/replay.mjs`; `DESIGNER-MOVES.md:121–123`).
- Playing invokes the same public action routes as a manual/API operation and maintains replay position/status. Replay output is a separate audit record, not a rewrite of the source script.
- `actionScriptFromAudit()` includes only completed recognized public actions (`design-history/replay.mjs`).
- A registered design route uses `molarium.registered-design-route/v1`. It defines the allowed coordinate boundary, hash-pinned hit, ordered graph edits, and locked evaluation state. It is input to a prospective workflow, not a history ledger (`design-history/structures/design-route.mjs`).
- The campaign ledger stores content-addressed structures/scripts, hash-chained events, decisions, commits, branches, and finalization state. Finalized entities are immutable (`design-history/ledger.mjs`).

Boundaries and records.

- Boundary: no additional scientific engine; scripts call the installed Chemist Actions boundary and therefore cross into the same workers as the invoked actions.
- Exports: action-script JSON, replay audit, and design campaign/standalone-viewer records where their specific UI or utility provides download/build output.
- Campaign actor types in the implementation are human, agent, system, and import (`design-history/ledger.mjs:28`).

Failure behavior and verified limits.

- Schema, action, binding, hash, or referenced-content mismatch stops validation/replay. The route loader rejects a campaign ledger, and the ledger verifier rejects a registered route. A registered story additionally verifies its source SHA-256 before installation (`design-history/structures/design-route.test.mjs`; `app.js:13983–14002`).
- Import is destructive to the unsaved in-memory scene by design; export first if it must be retained.
- Hash chains detect record mutation but do not authenticate an author, establish scientific truth, or replace a signature (`designhistory/README.md:131–144`).
- Scripts cannot bypass public action validation with direct coordinates or arbitrary JavaScript.

B.20 Enumeration development modules

Responsibility. Represent bounded topology-edit plans, execute them through the public API, describe graph-lineage disruption, and hold small development catalogues/panels.

Status and entry points. Repository tooling/development, not a general current workbench enumeration UI. Current implementation contains three development transformations and separates enumeration from docking (`enumerations/README.md:1–51`).

Inputs, outputs, state, and boundary.

- An action plan contains supported atom/bond/charge operations and explicit Finish boundaries. Validation enforces supported heavy elements, bond orders, charges, and required transaction completion (`enumerations/action-plan.mjs:14-59`).
- Execution resolves step aliases, calls only the public Chemist Actions API, and returns an execution audit plus final inspection (`enumerations/action-plan.mjs:62-105`).
- Edit-difficulty output describes graph-lineage disruption; it is not an affinity, synthesis, or MCS score (`enumerations/editedifficulty.mjs:57-130`).
- Pose assessment produces conservative flags for downstream review (`enumerations/pose-assessment.mjs:1-26`).

Records, failures, and limits.

- Development catalogues and preregistered high-disruption panels are files in `enumerations/`; Node validation includes a 7KPAspecific catalogue path (`enumerations/catalogue.mjs`).
- Invalid plans fail before execution. Runtime chemistry failures are retained in the execution audit through the public API.
- Combinatorial enumeration, general library management, global pose transfer, and automated campaign selection remain development/proposed work rather than normal UI functions (`enumerations/README.md:38-51`).

B.21 Validation, tests, and maintainer controls

Responsibility. Maintain versioned scientific reference records, run deterministic unit/integration/browser checks, and display the validation ledger in the application information dialog.

Entry points.

- The Validation project panel lazy-loads `validation/dashboard.mjs` (`app.js:13895-13911`).
- `package.json` defines unit, scientific, browser, build, and specialized validation scripts (`package.json:6-93`).
- CI builds the production distribution and runs scientific and browser suites (`.github/workflows/ci.yml:18-76`).

Inputs, outputs, state, and boundary.

- Registry v0.2 separates targets, reference systems, cases, poses, assertions, outcomes, and hashes. Versions are append-only; failed cases remain records (`validation/README.md:1-26`).
- Dashboard input is the checked-in registry; output is a read-only rendered summary. Failure renders "Validation ledger unavailable" rather than hiding the panel (`app.js:13895-13902`).
- Browser tests may use the separately enabled test API. Maintainer scripts use Node/Bun and, for browser suites, a browser process.

Records, failures, and limits.

- At inspection, registry validation reported 25 cases, 18 reference systems, 15 targets, and 5 crystal-scored cases; these are repository records, not an external certification (`README.md:368-387`).
- Registry hashes establish internal integrity. They do not make all cases independent or establish broad accuracy outside the represented systems.
- The main workbench dashboard is informational; it does not launch validation jobs or alter the registry.

B.22 Export and persistence matrix

Table 8: Export and persistence matrix.

Record	Produced by	Format	Contains	Does not contain
Current coordinates	Export	XYZ	Current displayed molecule coordinates	Full topology metadata, history, or calculation provenance
Conformer ensemble	Result export	SDF/V2000	Exportable conformer coordinates and molecular graph	Full session history and unrelated calculation state
Protein structure	Protein export	PDB	Imported/predicted protein coordinates and supported annotations	Complete application/session provenance
Preparation report	Preparation inspector	JSON	Options, issues, audit/ready status	A production-ready preparation guarantee
Docking labbook	Docking results	JSON and Markdown	Settings, mapping, candidates, failures, hashes, selection	A cryptographic author identity or binding free energy
Designer script/replay	Designer controls	JSON	Public actions, bindings, replay/audit data	Arbitrary code, direct coordinate injection, complete UI state
Canvas image	Save	PNG	Current 3D rendering	Molecularly reusable coordinates/topology
Share link	Share	URL	Current URL or registered story permalink	Unsaved molecule or session serialization
Chemist action history	API inspect/export path	JSON/action script	Bounded recognized call audit	Persistence across reload unless explicitly exported

There is no implemented autosave, database, browser project store, generic trajectory-file export, or one-click complete-session archive.

B.23 End-to-end operating sequence

The following sequence summarizes the current operating pattern. It is intentionally conservative: state-changing actions are explicit, unsupported chemistry fails rather than being silently repaired, and records needed beyond the current browser session must be exported.

A. Start from a blank canvas.

1. Open the application with `?blank` and confirm that the viewer is empty. Without `?blank`, the bundled launch molecule loads automatically (`app.js:13938-13974`).
2. If network provenance matters, inspect the network-policy panel. In Local Lab, PDB/CCD/MSA controls that require external services are unavailable.

B. Load a structure.

1. Choose one supported input path:
 - paste XYZ or PDB text;
 - upload an XYZ or PDB file;
 - enter a SMILES string or PDB identifier in connected mode; or
 - select a prepared example.
2. Check atom count, formula, charge, components, and the 3D graph. For a ligand, inspect the synchronized 2D depiction. Ordinary `.mol` upload should not be treated as a supported ingestion path because the current upload router does not invoke the implemented V2000 parser.

3. For a ligand, inspect pH and protonation candidates and apply the intended state before topology editing or parameterization.
4. For a PDB, open preparation details, select the applicable policies, preview the result, inspect every blocker or warning, and download the preparation report. Apply/parameterize only if the bounded workflow matches the intended use; the report remains explicitly non-production-ready.

C. Inspect and edit.

1. Use Components and residue/pocket focus to isolate the intended working region. Representation and contact settings are view state.
2. Enter Build. Select atoms in 2D or 3D and apply supported element, bond, charge, hydrogen, stereochemistry, fragment, or geometry operations.
3. For graph chemistry, keep related edits within the pending transaction. Choose Finish chemistry to sanitize, reconcile, and polish the graph and create one undo point. If validation fails, correct the pending edit or Discard to restore the starting graph.
4. Use Undo/Redo for committed changes. If a reference pose is required, capture it before the edit that creates the analogue and retain explicit stable/core/contact selections as needed.

D. Calculate or maintain a pose.

1. Confirm that no chemistry transaction is pending.
2. For an ordinary calculation, enter Run and select a compatible method and task:
 - RDKit: energy, geometry, or short dynamics;
 - Direct WebGPU: energy, geometry, or bounded dynamics;
 - WebGPU ensemble: dynamics or conformer search; or
 - ANI-2x: energy or geometry for an eligible molecule.
3. Set steps, solvent, constraints, replicas or conformer protocol, and saved frames as appropriate. Run the calculation and inspect engine/model identity, energy, convergence, timing, frames, and replica/conformer summaries. On failure, inspect the returned notice before retrying.
4. For reference-guided analogue docking, use Build: verify the captured reference, choose Pose propagation or an Expert core, review contact mappings, and run the constrained search. Inspect distinct candidates and the labbook. Coordinates change only when a selected candidate is explicitly applied.

E. Export and replay.

1. Export the chemically reusable record needed by the next step: current XYZ, conformer SDF, protein PDB, preparation report, or docking JSON/Markdown. A PNG is a visual record rather than a reusable molecular state.
2. For reproducible UI/API operations, export a Designer action script or campaign/replay record. An action script contains public operations, not a complete binary session snapshot.
3. Before replay, export any unsaved work because importing a script clears the scene. Restart the script, Play or step through it, verify the replay audit and resulting structure, and explicitly export any new records that must persist.
4. Before closing or reloading, export every needed record. The current browser session otherwise has no automatic persistence.

B.24 Failure and recovery rules

- **Input error:** the previous scene normally remains. Correct the text, file type, identifier, or network condition and retry.
- **Pending chemistry error:** edit validation keeps the transaction open. Correct it or Discard; calculations and docking remain blocked while chemistry is pending.
- **Calculation error:** the current graph remains; coordinates are applied only after a successful worker return. Inspect `lastCalculation.error`, correct eligibility, options, or assets, and retry (`app.js:12552-12738`).
- **Worker crash or message error:** pending requests for that worker reject, the instance is terminated, and a later request creates a fresh worker (`app.js:10167-10205`).

- **Asset hash error:** the job stops. Restore the expected versioned asset or manifest rather than bypassing verification (`app.js:10033-10155`).
- **Docking infeasibility:** no pose is applied. Repair the reference, core, contact mapping, or chemistry and rerun; retain the negative labbook record when provenance matters.
- **Replay or hash error:** stop and confirm the schema, registered source path, and hash. A partially replayed session is not equivalent to a completed audit.
- **Preparation blocker:** do not force Apply. Change preparation policy or use a system that supports the missing chemistry.
- **Unsaved state loss:** there is no automatic recovery after Clear, page reload, or script import.

B.25 Current, experimental, proposed, and retired boundaries

Current browser behavior.

- Structure load from XYZ/PDB/SMILES/PDB identifier and prepared examples.
- Synchronized 3D/2D inspection, component and residue focus, explicit graph editing, chemistry transactions, and undo/redo.
- RDKit calculation; export of current coordinates, conformers, proteins, reports, docking labbooks, action scripts, and images.
- Chemist Actions, Designer replay, registered hash-verified stories, and the read-only validation dashboard.

Experimental behavior that is nevertheless executable.

- Browser PDB preparation/general Sage-Gasteiger protein parameterization.
- Direct WebGPU force-field calculation.
- STORMM homogeneous WebGPU ensemble dynamics/conformer search and Conformer Arena.
- ANI-2x browser inference.
- OpenFold prediction with an external MSA boundary.
- Reference-guided docking, contact remapping, ring release, bounded sidechain alternatives, and induced-fit local optimization.
- Design-campaign/history records and standalone viewers, within their stated integrity limits.

Repository tooling/development only.

- Enumeration action-plan/catalogue modules and preregistered high-disruption panels.
- Browser test API, registry validators, scientific benchmarks, pose-review/build scripts, and CI commands.
- OpenMM as an internal/reference worker rather than a selectable current Run method.

Proposed or unavailable.

- General remote CUDA execution is not an installed runtime path.
- GFN/xTB is unavailable in the current workbench (`README.md:389-408`).
- General combinatorial enumeration and automated campaign selection are not current UI workflows.
- Missing-loop construction, membrane/metal/covalent-ligand preparation, periodic explicit solvent, PME, binding free energy, global docking, and production-scale trajectories are not implemented as end-user workflows.

Retired or compatibility-only.

- The OpenMM binary retains the older MolariumFF entry only for compatibility; it is not exposed (`openmm/README.md:13-15`).
- free-local cleanup remains an explicit older option; current reference-guided operation defaults to preserving the reference relationship (`index.html:287-294`).
- Git history shows the docking name change to ConstraintDock (`f9c31db`), the addition of synchronized 2D editing (`85d0b4a`), design history (`0639144`), Chemist Actions/replay (`0a7eeb3`), STORMM display-only alignment (`5621b8e`), manual hydrogen bonds (`c879077`), and the validation dashboard (`f8acdeb`). Earlier draft benchmark manifests and narrative prototypes are not current runtime contracts.

B.26 Verification performed for this manual

The following current repository checks passed during inspection:

- node chemist-actions.test.mjs
 - node docking/test.mjs
 - node docking/manual-hbond.test.mjs
 - node docking/restraint-biased-search.test.mjs
 - node docking/ring-conformer-generator.test.mjs
 - node docking/sidechain-rotamers.test.mjs
 - node enumerations/enumerations.test.mjs
 - node design-history/design-history.test.mjs
 - node design-history/replay-audit.test.mjs
 - node design-history/interface-story.test.mjs
 - node design-history/viewer/model.test.mjs
 - node design-history/structure-viewer/timeline.test.mjs
 - node design-history/structures/design-route.test.mjs
 - node validation/validate-registry.mjs
 - node validation/dashboard.test.mjs
- These checks establish that the inspected contracts pass their repository tests. They do not expand the operating limits listed above.

References

- [1] Bran, A. M.; Cox, S.; Schilter, O.; Baldassari, C.; White, A. D.; Schwaller, P. Augmenting Large Language Models with Chemistry Tools. *Nature Machine Intelligence* **2024**, *6*, 525–535.
- [2] Boiko, D. A.; MacKnight, R.; Kline, B.; Gomes, G. Autonomous Chemical Research with Large Language Models. *Nature* **2023**, *624*, 570–578.
- [3] Pham, T. D.; Tanikanti, A.; Keçeli, M. ChemGraph as an Agentic Framework for Computational Chemistry Workflows. *Communications Chemistry* **2026**, *9*, 33.
- [4] MacKnight, R.; Boiko, D. A.; Regio, J. E.; Gallegos, L. C.; Neukomm, T. A.; et al. Rethinking Chemical Research in the Age of Large Language Models. *Nature Computational Science* **2025**, *5*, 715–726.
- [5] Haas, A.; Rossberg, A.; Schuff, D. L.; et al. Bringing the Web up to Speed with WebAssembly. *Proc. PLDI* **2017**, 185–200.
- [6] W3C GPU for the Web Working Group. *WebGPU Specification*, 2025.
- [7] Riniker, S.; Landrum, G. A. Better Informed Distance Geometry: Using What We Know to Improve Conformation Generation. *J. Chem. Inf. Model.* **2015**, *55*, 2562–2574.
- [8] Boothroyd, S.; Madin, O. C.; Mobley, D. L.; et al. Development and Benchmarking of Open Force Field 2.0.0: The Sage Small-Molecule Force Field. *J. Chem. Theory Comput.* **2023**, *19*, 3251–3275.
- [9] Eastman, P.; Galvelis, R.; Peláez, R. P.; et al. OpenMM 8: Molecular Dynamics Simulation with Machine Learning Potentials. *J. Phys. Chem. B* **2024**, *128*, 109–116.
- [10] Cerutti, D. S.; Wiewiora, R. P.; Boothroyd, S.; Sherman, W. STORMM: Structure and Topology Replica Molecular Mechanics for Chemical Simulations. *J. Chem. Phys.* **2024**, *161*, 032501.
- [11] Devereux, C.; Smith, J. S.; Huddleston, K. K.; et al. Extending the Applicability of the ANI Deep Learning Molecular Potential to Sulfur and Halogens. *J. Chem. Theory Comput.* **2020**, *16*, 4192–4202.

- [12] Ahdriz, G.; Bouatta, N.; Floristean, C.; et al. OpenFold: Retraining AlphaFold2 Yields New Insights into Its Learning Mechanisms and Capacity for Generalization. *Nature Methods* **2024**, *21*, 1514–1524.
- [13] Ryckaert, J.-P.; Ciccotti, G.; Berendsen, H. J. C. Numerical Integration of the Cartesian Equations of Motion of a System with Constraints: Molecular Dynamics of n-Alkanes. *J. Comput. Phys.* **1977**, *23*, 327–341.
- [14] Andersen, H. C. RATTLE: A “Velocity” Version of the SHAKE Algorithm for Molecular Dynamics Calculations. *J. Comput. Phys.* **1983**, *52*, 24–34.
- [15] Cohen-Boulakia, S.; Belhajjame, K.; Collin, O.; et al. Scientific Workflows for Computational Reproducibility in the Life Sciences: Status, Challenges and Opportunities. *Future Gener. Comput. Syst.* **2017**, *75*, 284–298.
- [16] Pritchard, N. J.; Wicenec, A. Formal Definition and Implementation of Reproducibility Tenets for Computational Workflows. *Future Gener. Comput. Syst.* **2025**, *166*, 107684.
- [17] Wilkinson, M. D.; Dumontier, M.; Aalbersberg, I. J.; et al. The FAIR Guiding Principles for Scientific Data Management and Stewardship. *Sci. Data* **2016**, *3*, 160018.
- [18] Bender, A.; Cortés-Ciriano, I. Artificial Intelligence in Drug Discovery: What Is Realistic, What Are Illusions? Part 1. *Drug Discov. Today* **2021**, *26*, 511–524.
- [19] Bender, A.; Cortés-Ciriano, I. Artificial Intelligence in Drug Discovery: What Is Realistic, What Are Illusions? Part 2. *Drug Discov. Today* **2021**, *26*, 1040–1052.
- [20] Wigh, D. S.; Goodman, J. M.; Lapkin, A. A. A Review of Molecular Representation in the Age of Machine Learning. *WIREs Comput. Mol. Sci.* **2022**, *12*, e1603.
- [21] Bainbridge, L. Ironies of Automation. *Automatica* **1983**, *19*, 775–779.
- [22] W3C GPU for the Web Working Group. *WebGPU Shading Language*, 2026. <https://www.w3.org/TR/WGSL/>.
- [23] Wang, S.; Witek, J.; Landrum, G. A.; Riniker, S. Improving Conformer Generation for Small Rings and Macrocycles Based on Distance Geometry and Experimental Torsional-Angle Preferences. *J. Chem. Inf. Model.* **2020**, *60*, 2044–2058.
- [24] Halgren, T. A. Merck Molecular Force Field. I. Basis, Form, Scope, Parameterization, and Performance of MMFF94. *J. Comput. Chem.* **1996**, *17*, 490–519.
- [25] Rappé, A. K.; Casewit, C. J.; Colwell, K. S.; Goddard, W. A.; Skiff, W. M. UFF, a Full Periodic Table Force Field for Molecular Mechanics and Molecular Dynamics Simulations. *J. Am. Chem. Soc.* **1992**, *114*, 10024–10035.
- [26] Gasteiger, J.; Marsili, M. Iterative Partial Equalization of Orbital Electronegativity—A Rapid Access to Atomic Charges. *Tetrahedron* **1980**, *36*, 3219–3228.
- [27] Eastman, P.; Swails, J.; Chodera, J. D.; et al. OpenMM 7: Rapid Development of High Performance Algorithms for Molecular Dynamics. *PLoS Comput. Biol.* **2017**, *13*, e1005659.
- [28] Onufriev, A.; Bashford, D.; Case, D. A. Exploring Protein Native States and Large-Scale Conformational Changes with a Modified Generalized Born Model. *Proteins* **2004**, *55*, 383–394.
- [29] Glaser, J.; Nguyen, T. D.; Anderson, J. A.; et al. Strong Scaling of General-Purpose Molecular Dynamics Simulations on GPUs. *Comput. Phys. Commun.* **2015**, *192*, 97–107.

- [30] Hillig, R. C.; Sautier, B.; Schroeder, J.; et al. Discovery of Potent SOS1 Inhibitors That Block RAS Activation via Disruption of the RAS-SOS1 Interaction. *Proc. Natl. Acad. Sci. U.S.A.* **2019**, *116*, 2551–2560.